Modul 13

Alternative Datenbanken

Ausgewählte (aktuelle) Entwicklungen

Josef Altmann





Der vorliegende Foliensatz basiert auf:

Sadalage, M., Fowler, M.: *NoSQL Distilled – A Brief Guide to the Emerging World of Polyglott Persistence*, Addision-Wesley (2013)

Perkins, L., Redmond, E.: *Seven Databases in Seven Weeks: A Guide to Modern Databases and the NoSQL Movement*, O'Reilly, 2. Auflage (2018)

Kleppmann M.: Datenintensive Anwendungen designen, O'Reilly, 1. Auflage (2019)

Kudraß, T., Störl, U.: NoSQL-Datenbanken, Hanser, 1. Auflage (2027)

Überblick

- Motivation
- Big Data
- NoSQL
- Zusammenfassung

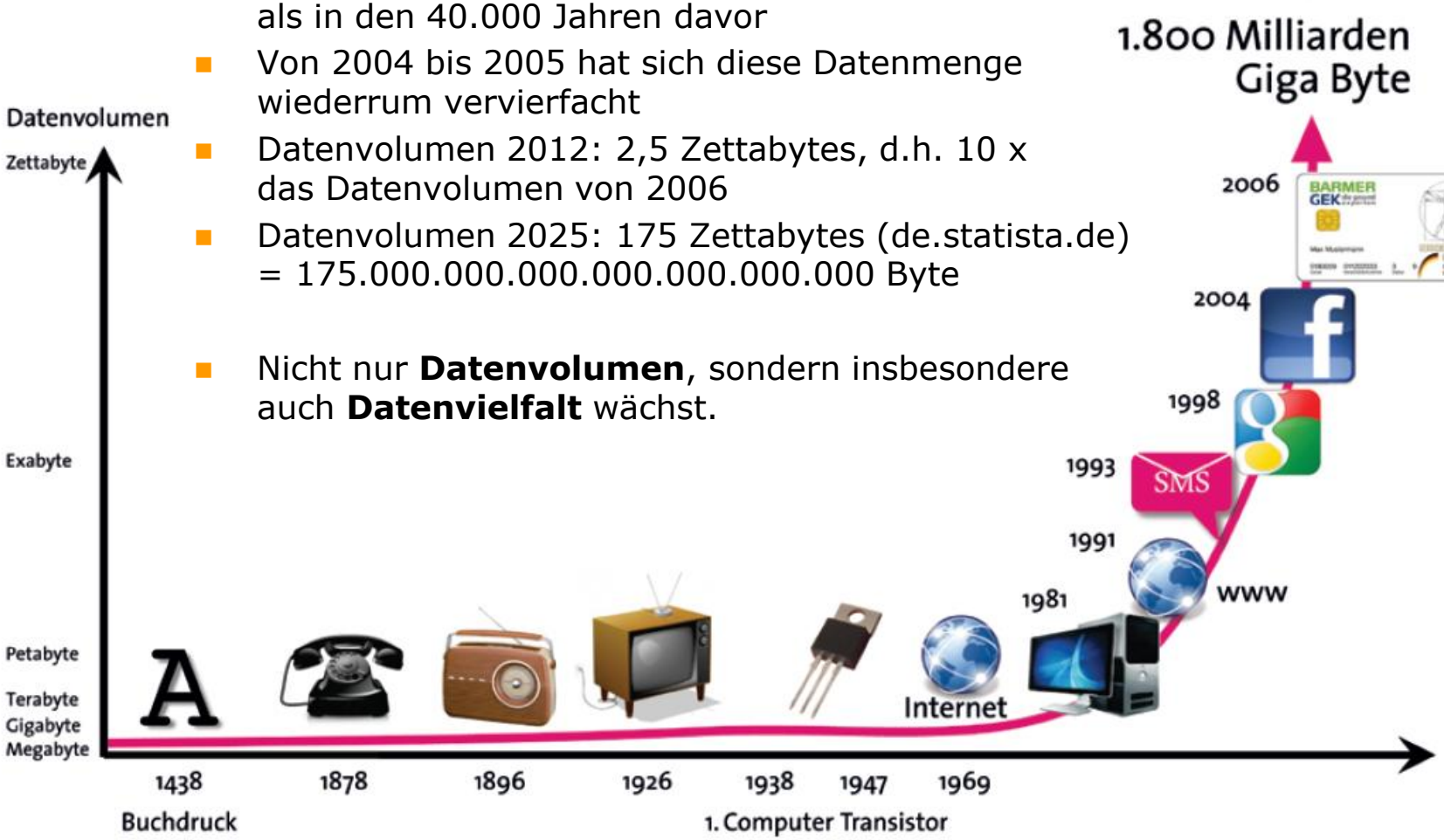
© 2026

Exponentielles Datenwachstum



- Von 2000 bis 2002 sind mehr Daten generiert worden, als in den 40.000 Jahren davor
- Von 2004 bis 2005 hat sich diese Datenmenge wiederrum vervierfacht
- Datenvolumen 2012: 2,5 Zettabytes, d.h. 10 x das Datenvolumen von 2006
- Datenvolumen 2025: 175 Zettabytes (de.statista.de) = 175.000.000.000.000.000.000 Byte

- Nicht nur **Datenvolumen**, sondern insbesondere auch **Datenvielfalt** wächst.



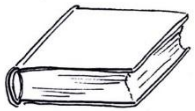
BITCOM: Big Data im Praxiseinsatz – Szenarien, Beispiele, Effekte (2012)
https://www.bitkom.org/

Was ist ein Zettabyte?

2,5 **Zettabytes** = 2,5 * 10^{21} Bytes =
2,5 Tausend Exabyte
2,5 Millionen **Petabyte** =
2,5 Milliarden Terabyte =
2,5 Billionen **Gigabytes**

Wieviel ist das?

1 Buchseite = 2.500 Byte

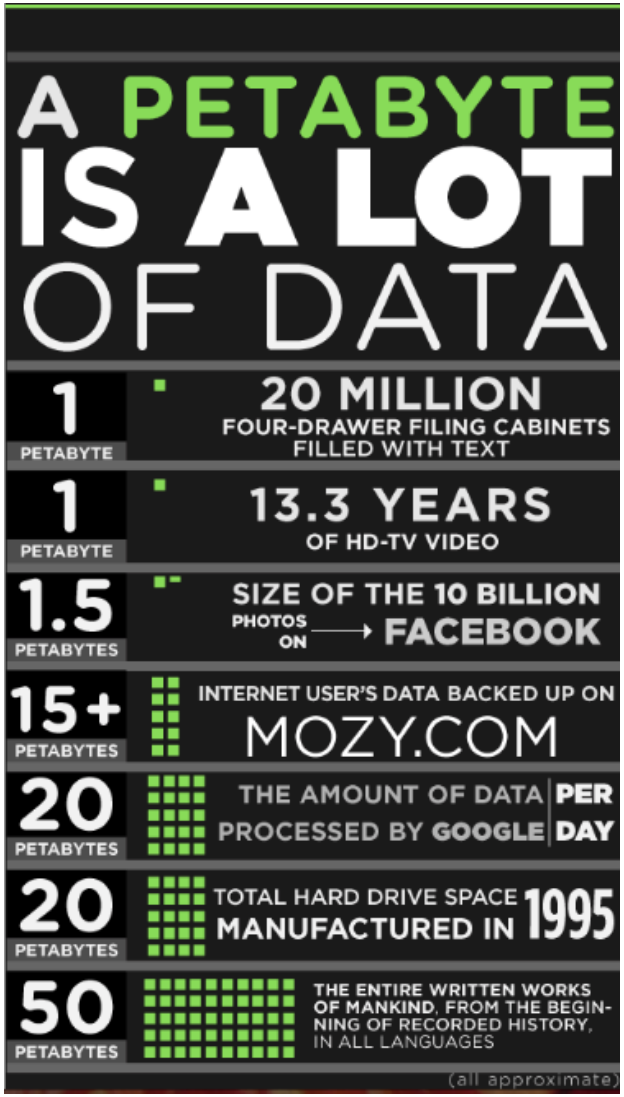


1 Trilliarde Buchseiten



Wie hoch wäre der Stapel?

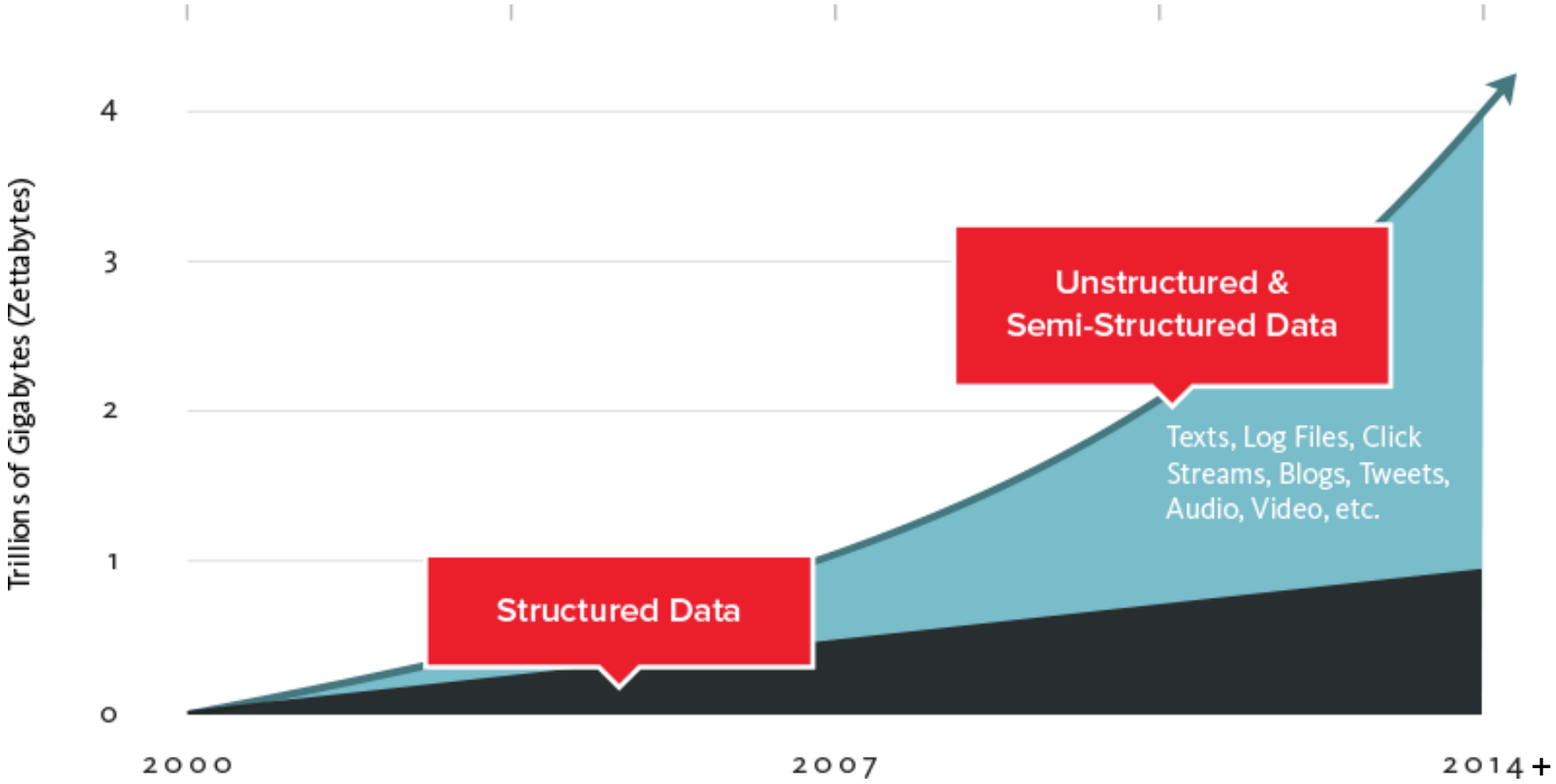
600 Mrd. Kilometer = 6 x Entfernung Sonne zur Erde!



© 2026

Datenwachstum

Unstrukturierte/Semi-strukturierte Daten



http://www.couchbase.com/why-nosql/nosql-database

Überblick

- Motivation
- Big Data
- NoSQL
- Zusammenfassung

Big Data
V-Eigenschaften

Volume
Immer größere Datenmengen müssen gespeichert und verarbeitet werden.

Value
Die vielen polystrukturierten Daten müssen genützt werden, um entscheidungsrelevante Erkenntnisse zu gewinnen und die Innovations- und Wettbewerbsfähigkeit zu steigern.

Velocity
Daten werden in großer und exponentiell wachsender Geschwindigkeit erzeugt.



Variety
Strukturierte, semi-strukturierte und unstrukturierte Daten beinhalten wertvolle Informationen.



Veracity
Die Vertrauenswürdigkeit (Wahrhaftigkeit) der Daten ist oft nur begrenzt gegeben.

© 2026

Big Data
Definitionen

McKinsey (2012):
Big data refers **to datasets whose size is beyond the ability of typical database software tools to capture, store, manage, and analyze**. This definition is intensionally subjective and **incorporates a moving definition of how big a dataset need to be** in order to be considered big data. … We assume that, as technology advances over time, the size of datasets that qualify as big data will also increase.

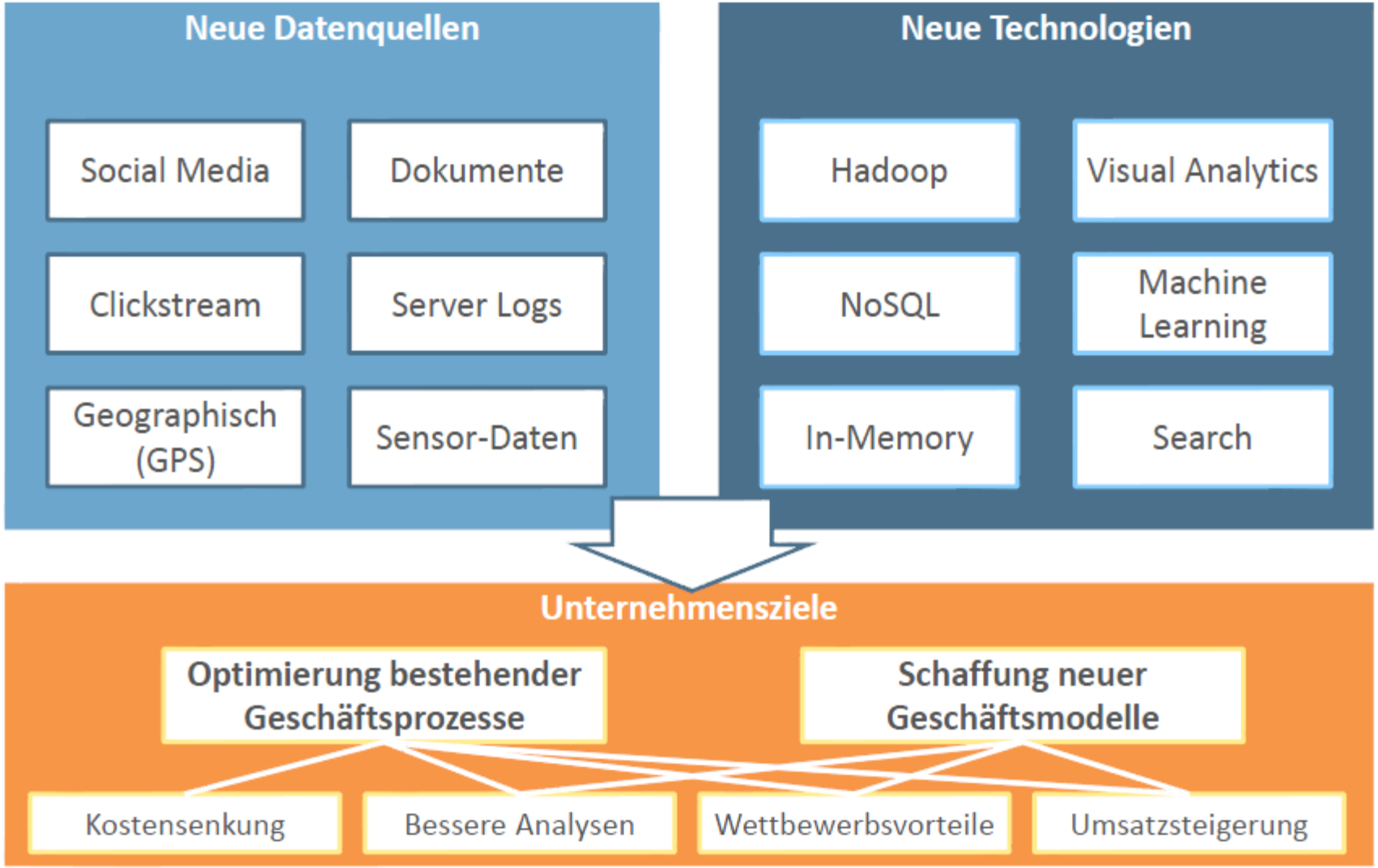
BITCOM (2012):
Big Data bezeichnet den Einsatz **großer Datenmengen aus vielfältigen Quellen** mit einer hohen Verarbeitungsgeschwindigkeit zur **Erzeugung wirtschaftlichen Nutzens.**

Datenkompetenz, Hanser Verlag (2023):
Big Data bezeichnet Datenmengen, die wegen ihrer Größe (Volume), der geforderten Verarbeitungsgeschwindigkeit (Velocity) oder aufgrund ihrer Vielfalt (Variety) nicht auf einem einzelnen Rechnersystem gehalten werden können, sondern **parallel auf mehreren Server verteilt** werden müssen.

© 2026

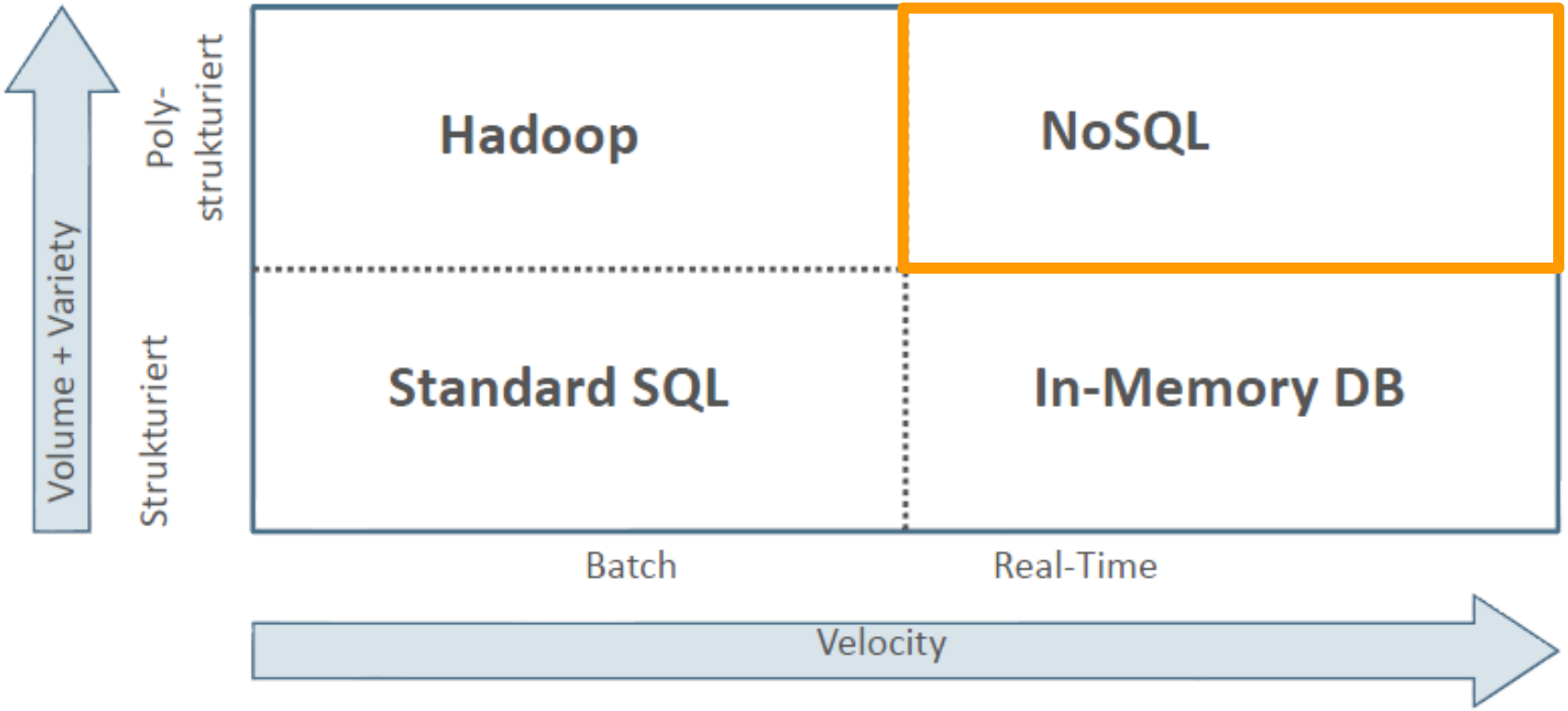
Big Data

Potentiale und Herausforderungen



© 2026

Big Data
Technologien



Big Data Technologien, Bitkom Leitfaden, 2014
http://www.bitkom.org

Überblick



- Motivation
- Big Data
- NoSQL
- Zusammenfassung

Einsatz von NoSQL ist dann zu überlegen, wenn folgende Fragen auftreten:

- Haben Sie Probleme mit der Anzahl an schreibenden Operationen pro Sekunde?
- Wie schnell bzw. wie oft können und wollen Sie ihre Reports erstellen?
- Wie gut passt ihr logisches Datenmodell wirklich zum relationalen Modell?

NoSQL-Systeme sind hochspezialisierte Lösungen für meist exakt ein großes Problem beim Umgang mit sehr großen, sehr gut verfügbaren oder sehr schnell benötigten Daten.

© 2026

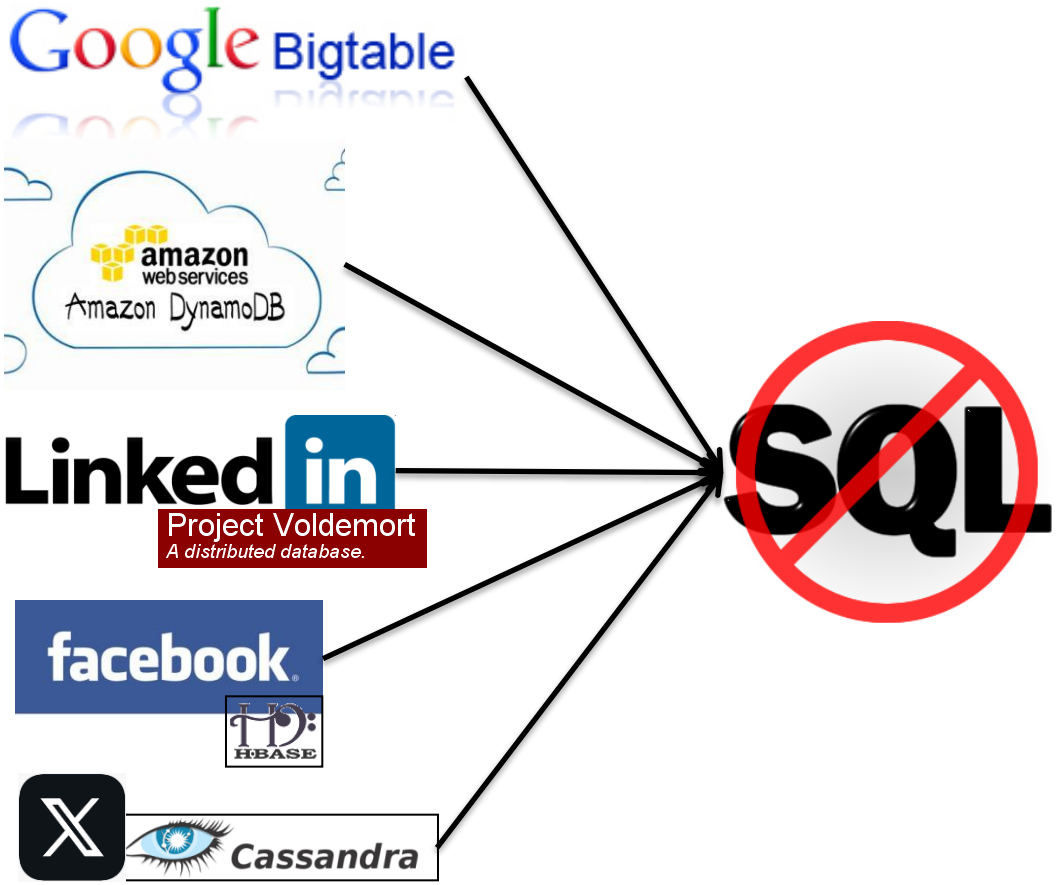
NoSQL

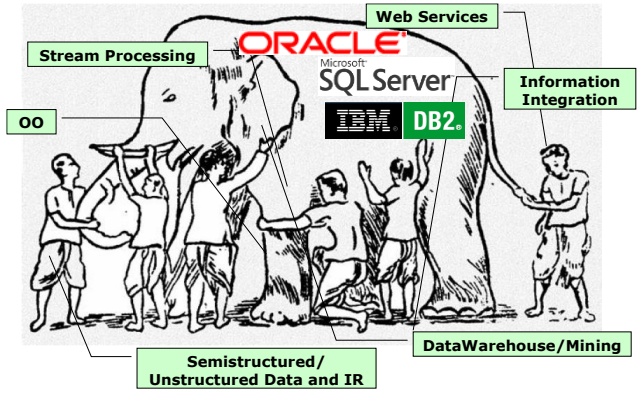
Das aktuelle Datenbank-Buzzword



"How to write a CV" Geek & Poke Cartoon
http://geekandpoke.typepad.com/geekandpoke/2011/01/nosql.html

Aktuell genutzte Datenspeicher ...





The end of an era???

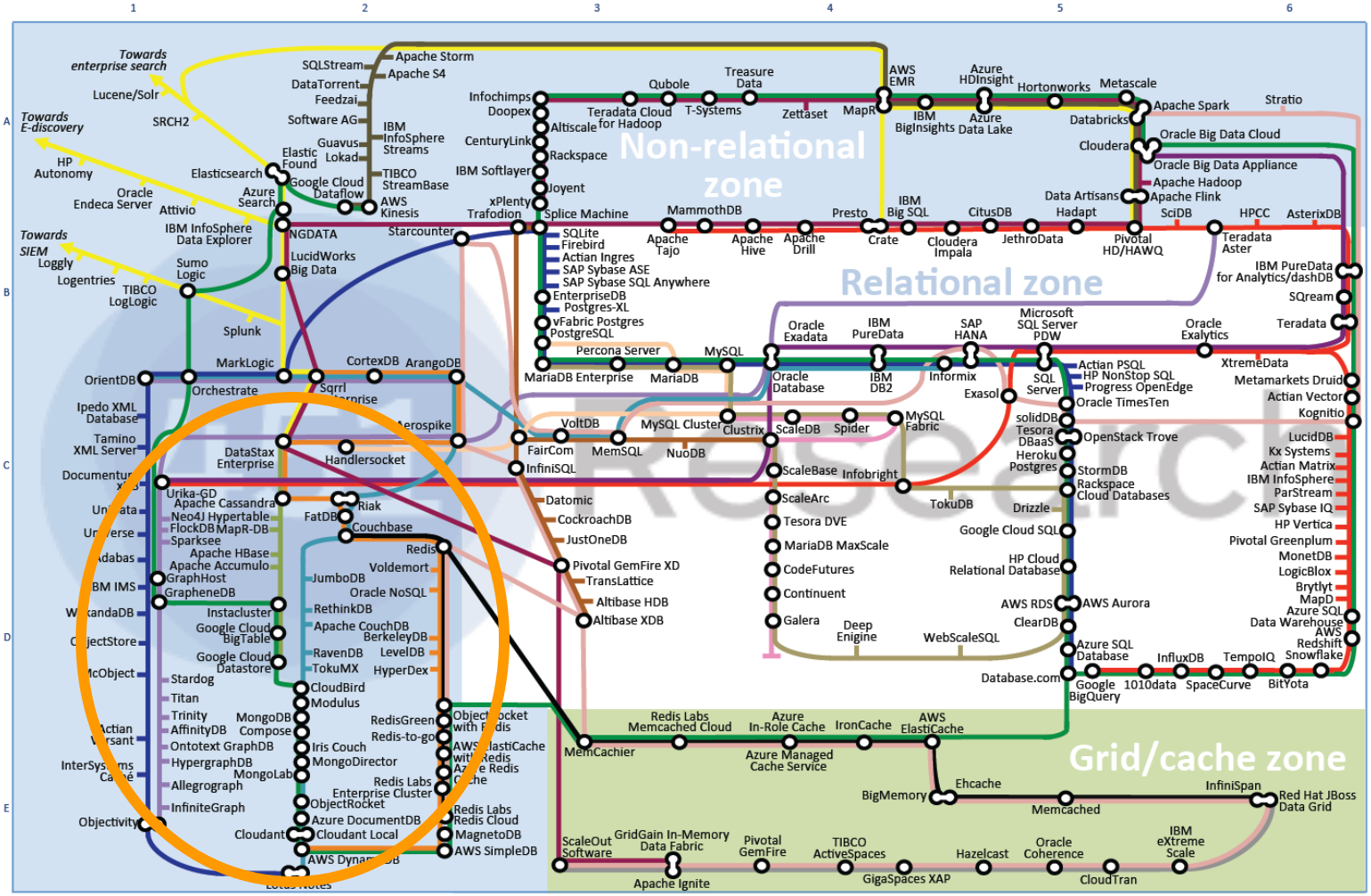
TPC-C Performance

- **Elephant**
  - 850 TPS
- **NoSQL-Store**
  - 70.416 TPS

→ **Factor of 82!**

M. Stonebraker, S. Madden, D. J. Abadi, S. Harizopoulos, N. Hachem, and P. Helland. 2007. The end of an architectural era: (it's time for a complete rewrite). In Proc. of VLDB '07. 1150-1160.

Aktuell genutzte Datenspeicher ...



Relationale Datenbanken

- Klassische relationale Datenbanken beherrschen die Datenbankwelt seit vielen Jahrzehnten
  - **Einfache Konzepte** (Tabellen, Spalten, Tupel)
  - **Logisches Datenmodell** mit **physischer Datenunabhängigkeit**
  - Klare Trennung von **Schema und Instanz**
  - **Zentrale**, integrierte **Datenhaltung**
  - Solide **mathematische Grundlagen** (Mengentheorie, Algebra)
  - Standardisierte Abfrage- und Manipulationssprache – **SQL**
  - Umsetzung von **ACID**-Eigenschaften (Atomarität, Konsistenz, Isolation, Dauerhaftigkeit)
  - …

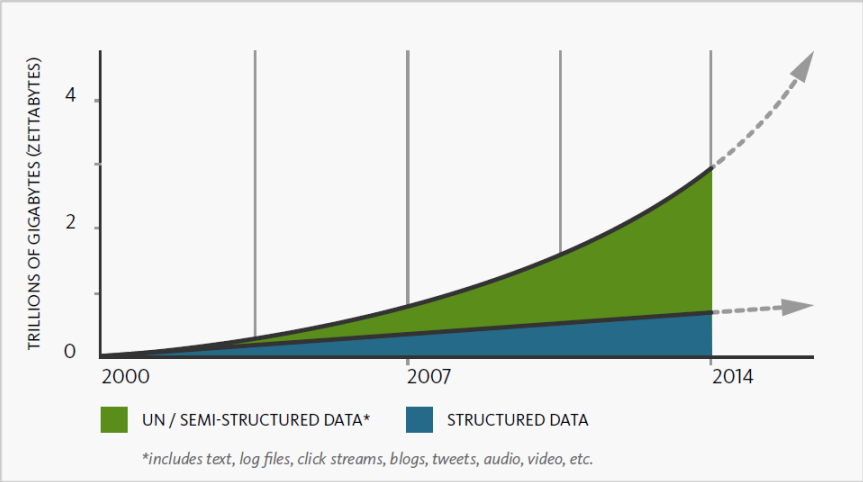
© 2026

Motivation - Warum NoSQL?

Web Daten

Relationales Paradigma und SQL sind für Web Daten nur eingeschränkt geeignet!

- Web Daten meist **unstrukturiert** bzw. **sem**
- **Einfache Operationen für Web** - bspw. für das von User Profiles, Freundelisten, Web Postings, Wikis,
  - Key Lookups – Wert zu Schlüssel finden
  - Lesen und Schreiben weniger Datensätze
  - Ausdrucksstärke von SQL nicht notwendig
    - → Einfache API (bspw. REST) genügt



http://www.couchbase.com/why-nosql/nosql-database

Produktkatalog

```
{
   "ProductId":"69451",
   "Type":"Smartphone",
   "Name":"S4",
   "Brand":"Samsung",
   "Features":{
           "Weight":499,
           "Colors":["black","blue"],
           "ScreenSize":12.7,
           "CameraResolution":"13 Megapixel"
   }
}
```

```
{
   "ProductId":"78462",
   "Type":"Shoe",
   "Name":"Timberland Classic",
   "Brand":"Timberland",
   "Weight":499,
   "Features":{
           "Weight":1400,
           "Sizes":[41,42,43,44,45],
           "Material":"Leather"
   }
}
```

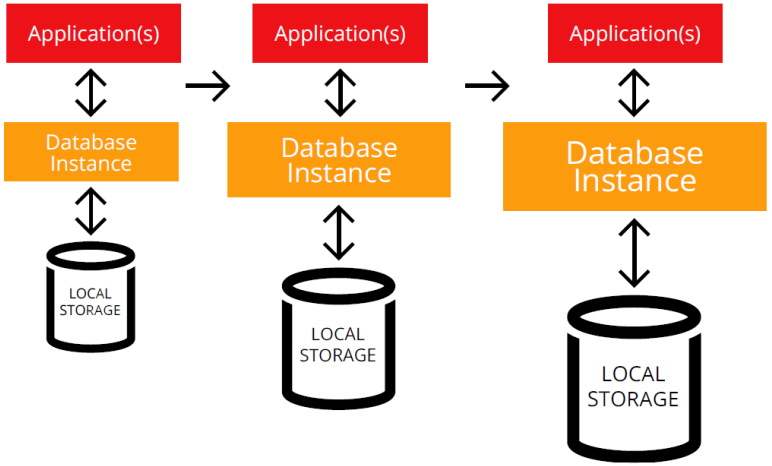
Motivation - Warum NoSQL?

Big Data und Big Users

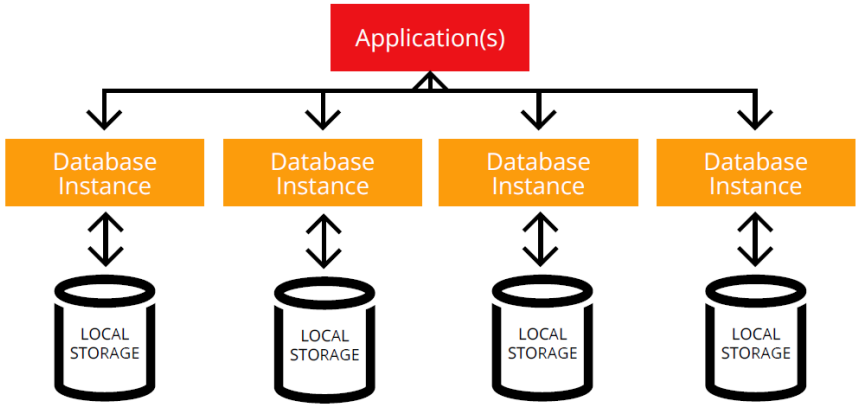
- Big Data und Big Users setzen Skalierbarkeit voraus
  → Verteilung von Daten und Benutzerlast notwendig!

- Zwei Strategien, um Skalierbarkeit zu erreichen:

- **Scale up** (vertikal):
  wenige, große Server
  → Shared Memory/Disk



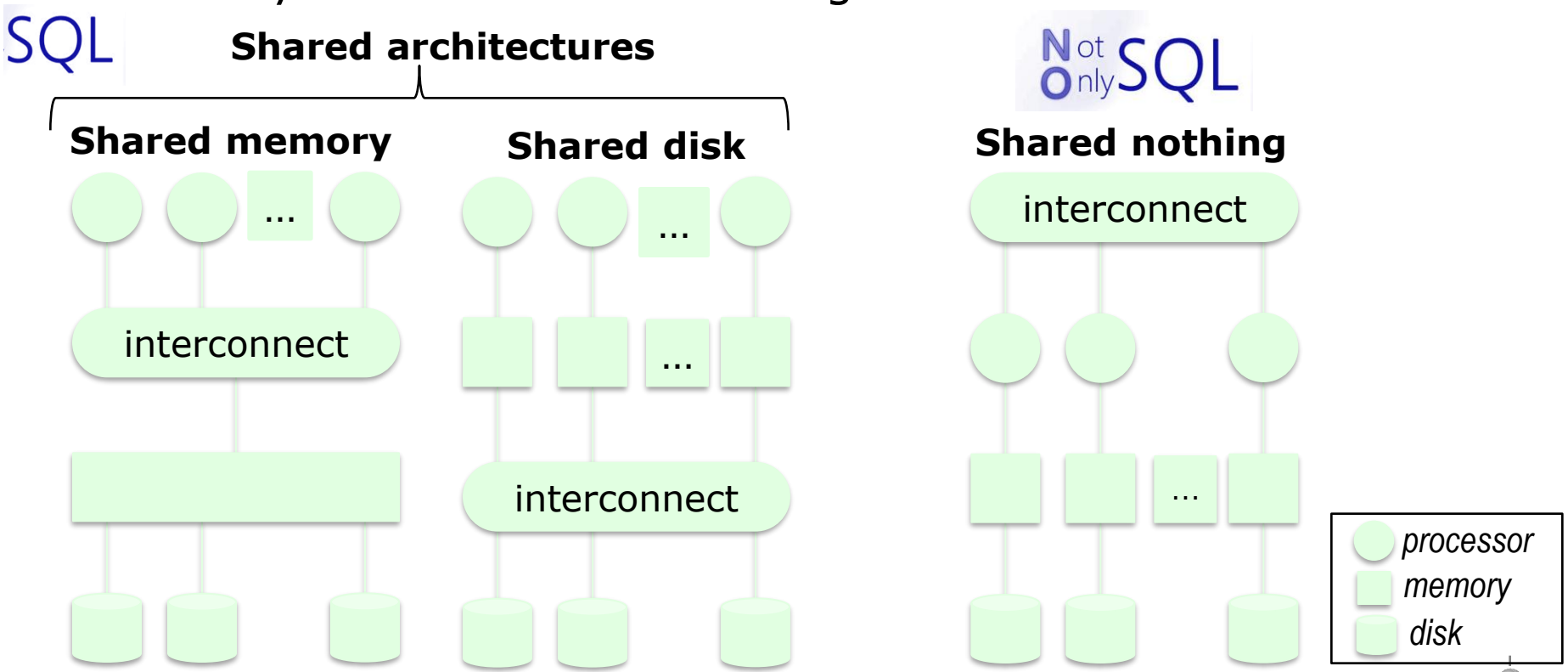
- **Scale out** (horizontal):
  viele, kleinere Server
  → verteiltes System/Replikation



Motivation - Warum NoSQL?

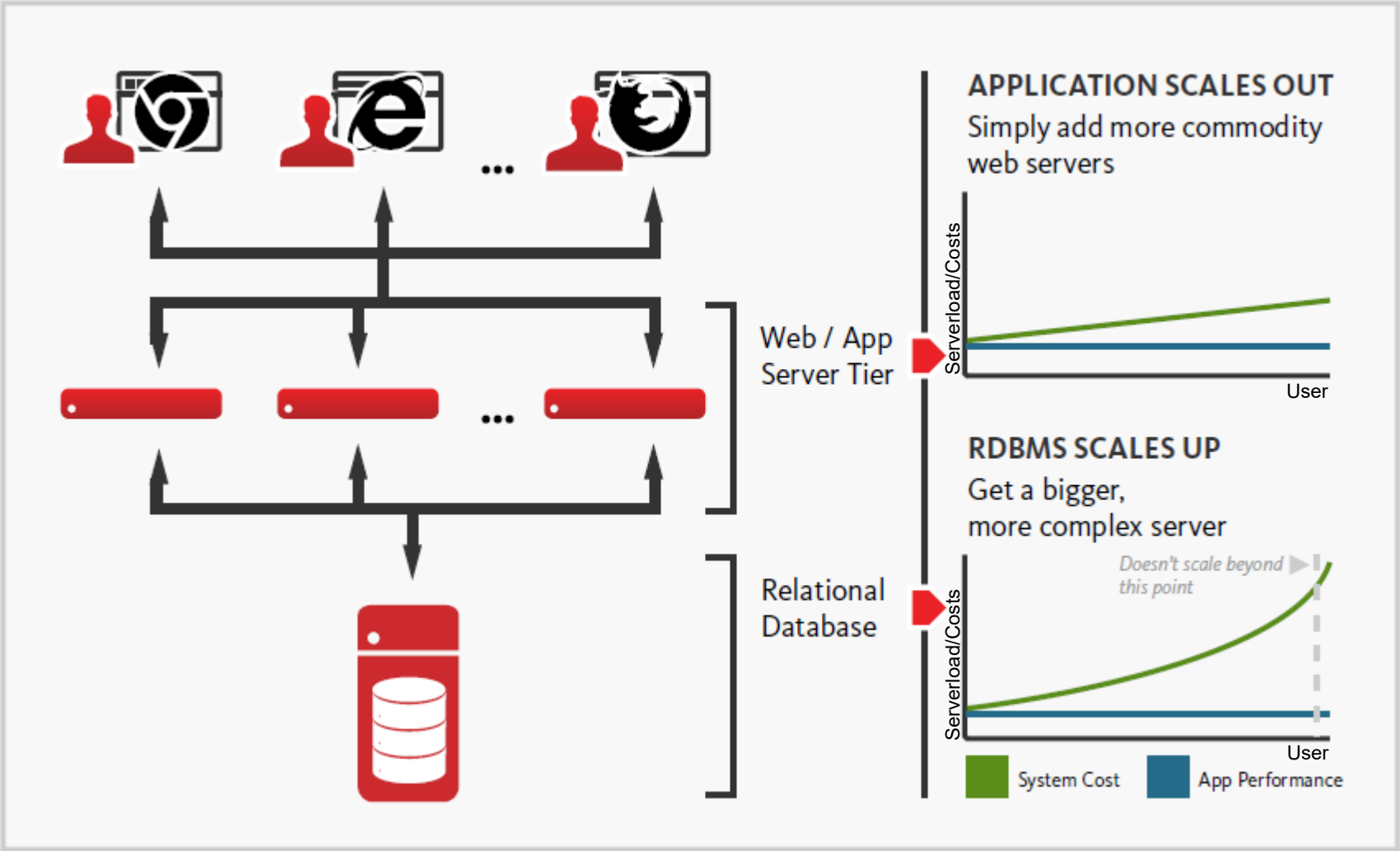
Big Data und Big Users

- **Shared architectures** (memory/disk) → eingeschränkte Skalierbarkeit durch gemeinsame Ressourcen
- **Shared nothing architectures** → hohe Skalierbarkeit, aber Synchronisation notwendig



Motivation - Warum NoSQL?

Scaling up Database Tier

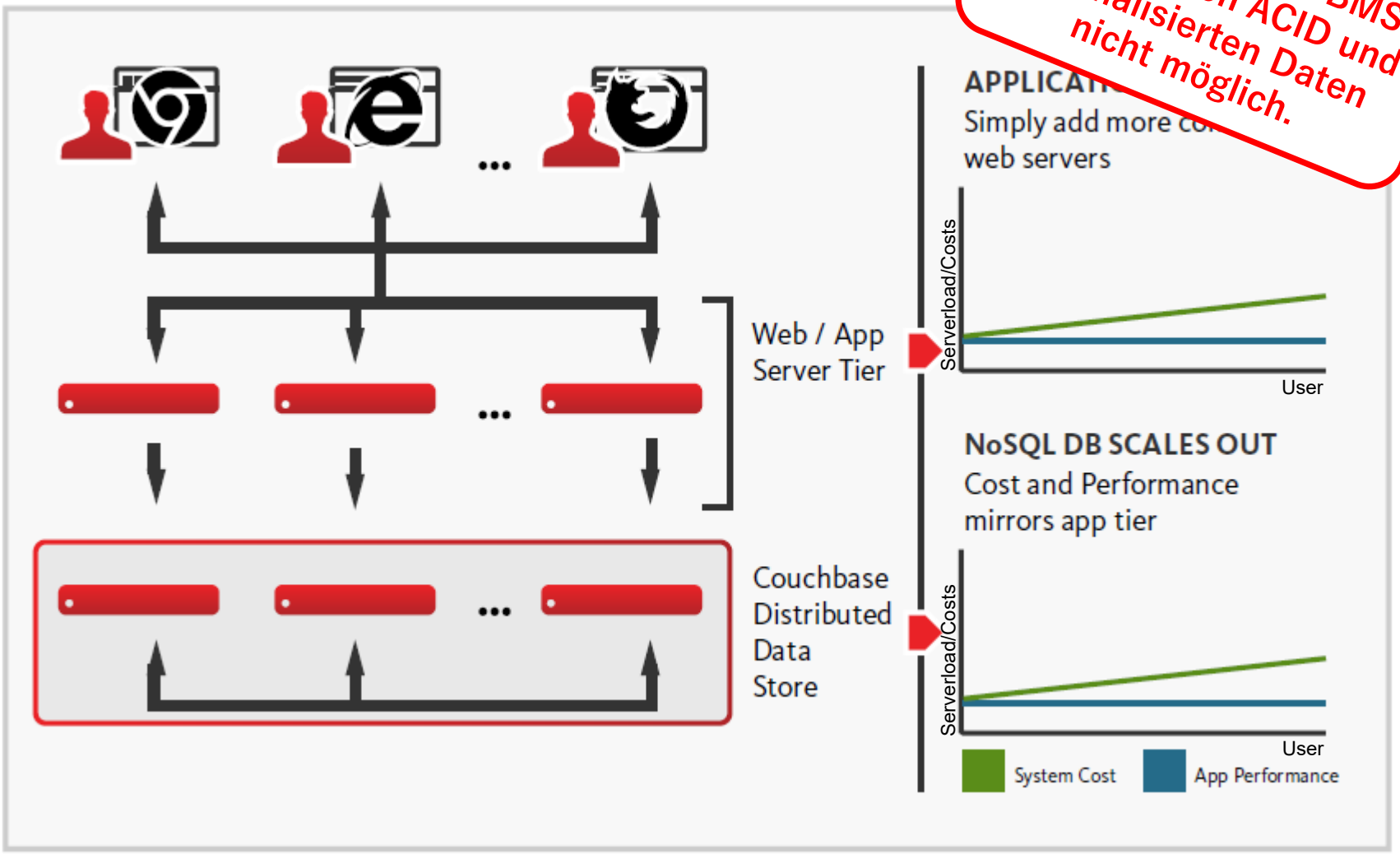


http://www.couchbase.com/nosql-resources/what-is-no-sql

Motivation - Warum NoSQL?

Scaling out Database Tier

Scale Out bei RDBMS aufgrund von ACID und normalisierten Daten nicht möglich.



http://www.couchbase.com/nosql-resources/what-is-no-sql

Eigenschaften von NoSQL-Datenbanken

- Nicht-relational
- „Kein explizites Schema“
  - Hinzufügen von Attributen zur Laufzeit

→ **NoSQL-Datenmodelle**

- Verfügbarkeit
  - Fehlertoleranz durch Replikation
- Horizontale Skalierbarkeit
  - Verteilte Architektur unterstützt Datenreplikation

→ **Skalierbarkeit**

- Abstriche bei der Konsistenz
  - Eventually Consistency
  - BASE anstatt ACID

→ **Konsistenz**

- Einfache API für Zugriff
  - keine JOINs

→ **MapReduce**

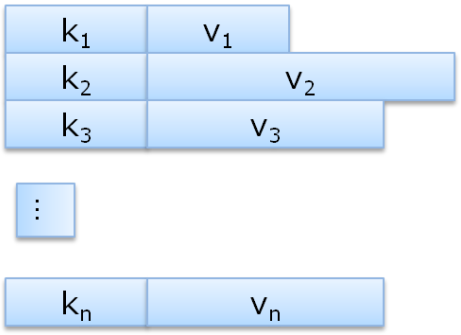
- Open Source

© 2026

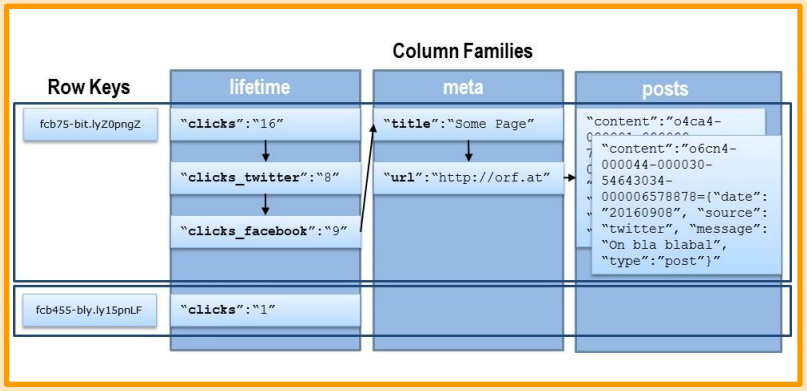
NoSQL-Datenmodelle

All Key-based!

Key-Value



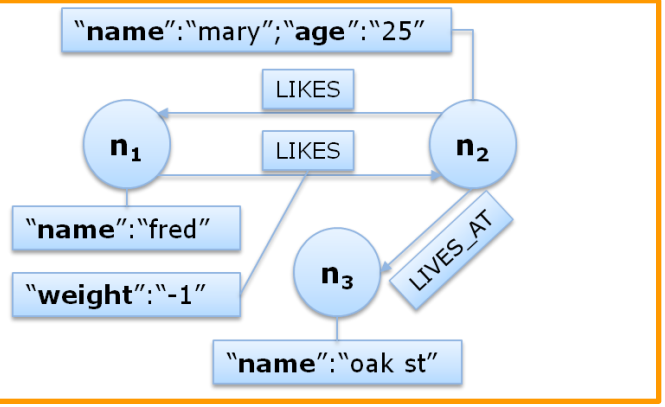
Wide-Column



Document



Graph-based

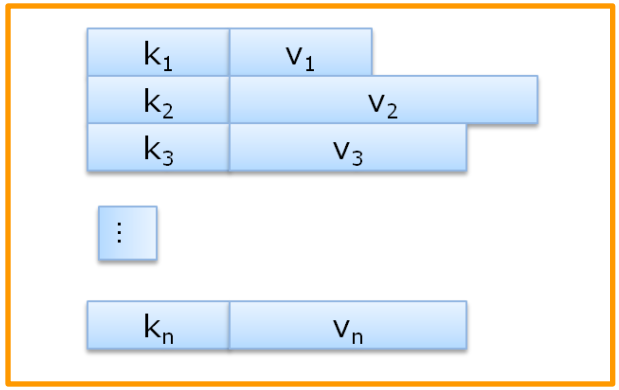


T. Haughey, The Role of Data Architecture in NOSQL, DAMA Day NY, Sept. 2011.
R. Cattell, *"Scalable SQL and NoSQL Data Stores", 2010.*

© 2026

NoSQL-Datenmodelle
Key/Value Stores

- Einfaches Datenmodell basierend auf **Schlüssel-Werte-Paaren** („the big hash table“)
- **Einfaches Interface**
  - `put (key, value)`
  - `get (key): value`
  - `delete (key)`
- **Datenspeicherung**
  - Werte (= Daten) werden auf Basis der benutzerdefinierten Schlüssel gespeichert
  - Werte entweder strukturlos (string) oder Sets/Listen/Hashes
- **Abfragen** basieren auf Schlüssel
  - Abfragen ohne Schlüssel schwierig
- **Indizes** können für Schlüssel definiert werden
  - Manche Systeme erlauben auch Sekundärindizes für Werte



NoSQL-Datenmodelle
Key/Value Stores

- Vorteile
  - Performanter, hochverfügbarer Zugriff auf Key/Value-Informationen
  - Gut skalierbar durch horizontale Partitionierung und Hashing
- Nachteile
  - kein vordefiniertes Schema, d.h. Anwendung verwaltet Schema
  - keine Integritätsbedingungen
  - keine/wenige Datentypen
  - eingeschränkte Abfragemöglichkeit: `get (key): value => only`
- Systeme







- Key-Value Cache
- Key-Value Store
- Eventually Consistent Key-Value Store
- Ordered Key-Value Store/Cache







… und viele mehr …

NoSQL-Datenmodelle

Key/Value Stores - Redis



Programmer-defined key!

- Basic Operations
  - `SET foo bar` (now bar is stored under the key foo)
  - `GET foo` (bar is returned)
  - `INCR foo` (assuming the value of foo to be 10; after `incr`, the value of foo is 11)
- Lists
  - `LPUSH myList a` (now the key myList holds one element 'a')
  - `LPUSH myList b` (now the key myList holds 'b,a')
  - `RPUSH myList c` (now the key myList holds 'b,a,c')
  - `LRANGE myList 0 1` ('b,a' is returned)
  - `LRANGE myList 0 -1` ('b,a,c' is returned)
- Sets (keine Duplikate)
  - `SADD mySet a` (now the key mySet holds one element 'a')
  - `SADD mySet b` (now the key mySet holds the elements 'a' and 'b')
  - `SMEMBERS mySet` ('a,b' or 'b,a' is returned)
  - `SADD mySet2 a`
  - `SINTER mySet mySet2` ('a' is returned, as being contained in both sets)

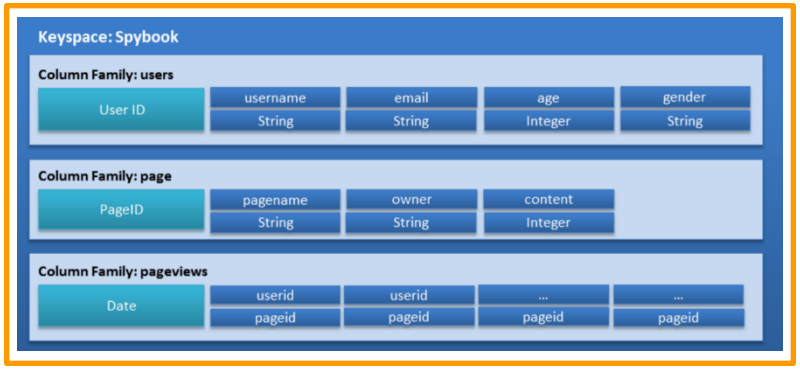
© 2026

NoSQL-Datenmodelle
Key/Value Stores

- **When to Use**
  - If data may be **encapsulated** within a **single value**, which may be identified by a **unique key**
  - Examples (caching of session data)
    - **Session Information**
      - **Key**: `sessionId`; **Value**: session information
    - **User Preferences**
      - **Key**: `userId` or `username`; **Value**: preferences such as `language`, ...
    - **Shopping Cart Data**
      - **Key**: `userID`; **Value**: shopping information
- **When Not to Use**
  - **Relationships** among **Data**
    - Relationships among different values are problematic
  - **Multi-operation Transactions**
    - Transactions spanning multiple keys are problematic
  - **Query by Data**
    - Queries on basis of the value do not perform well

NoSQL-Datenmodelle
Wide Column Stores

- **Zeilen** können **unterschiedliche Schemata** aufweisen
  - Festdefinierte Spaltenfamilien (Column Families) mit
  - variabler Anzahl von Spalten (Columns)
- **Interface**
  - `define (family)`
  - `insert (family,key,columns)`
  - `get (family,key): columns`



- **Datenspeicherung**
  - Variable Listen von Key-Value-Paaren (= Columns) innerhalb Column Families
  - Spaltenorientierte Speicherung
- **Abfragen** basieren auf Schlüssel und Column Family
- **Sekundärindex** pro Column Family meist durch Systeme unterstützt

NoSQL-Datenmodelle
Wide Column Stores

- Vorteile
  - Flexibles Schema
  - Hohe Skalierbarkeit bei steigender Anzahl von Spalten
- Systeme









… und viele mehr …

NoSQL-Datenmodelle

Wide Column Stores - Cassandra



- Basic Operations
  - ```
    CREATE COLUMN FAMILY Customer
    ...
    AND key_validation_class=UTF8Type
    AND column_metadata = [
    {column_name: city, validation_class: UTF8Type}
    {column_name: name, validation_class: UTF8Type}
    {column_name: web, validation_class: UTF8Type} ];
    ```
  - ```
    SET Customer['altmann']['city']='Boston';
    SET Customer['altmann']['name']='Josef Altmann';
    SET Customer['altmann']['web']='www.altmann.com';
    ```
  - ```
    GET Customer['altmann'];
    GET Customer['altmann']['web'];
    ```
  - ```
    DEL Customer['altmann']['city'];
    DEL Customer['altmann'];
    ```
  - ```
    SET Customer['altmann']['demo_access']='allowed' WITH
    ttl=2592000;
    ```
    ```

NoSQL-Datenmodelle

Wide Column Stores - Cassandra



- Casandra Query Language (CQL)
  - SQL-like query commands
  - BUT: no joins or subqueries; only simple where clauses

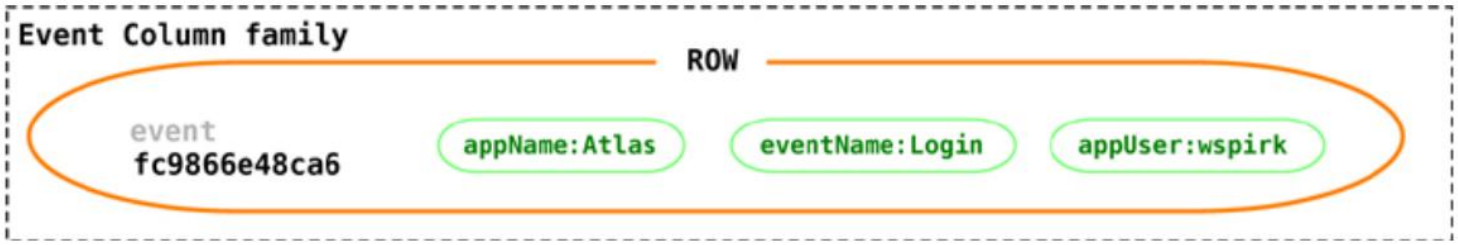
  - ```
    CREATE COLUMNFAMILY Customer (
     KEY varchar PRIMARY KEY,
     name varchar,
     city varchar,
     web varchar);
    ```

  - ```
    INSERT INTO Customer (KEY, name, city, web)
     VALUES        ('altmann','Josef Altmann', 'Boston',
                   'www.altmann.com');
    ```

  - ```
    SELECT * FROM Customer;
    ```
    ```

NoSQL-Datenmodelle
Wide Column Stores

- **When to Use**
  - If data fits well the "map of a map of a map structure", i.e., may be accessed by this pattern
  - Example
    - **Event Logging**
      Row key: `appname`
      Column family: `Event`
      Columns: `appName`, `eventName`, `appUser`



- **When Not to Use**
  - If **ACID transactions** are required
  - Queries with **aggregations** (SUM, AVG)

NoSQL-Datenmodelle
Document Stores

- **Key-Value-Store** für **semi-strukturierte Dokumente**
  - Dokumente repräsentieren hierarchische Datenstrukturen
  - Dokumentformate: JSON, XML, YAML oder RDF
- **Interface**
  - `set(key, document)`
  - `get(key): document`
  - `set(key, name, value)`
  - `get(key, name): value`



- **Datenspeicherung**
  - Schemafrei, d.h. Anwendung übernimmt Schemaverwaltung
  - Arrays, Listen, Zeiger und geschachtelte Dokumente
- **Abfragen** basieren auf Schlüssel (oder Attribut, wenn Index existiert)
- **Primär- und Sekundärindizes** werden unterstützt

NoSQL-Datenmodelle
Document Stores

- Vorteile
  - Schemalos (implizites Schema)
  - Anwendung ist für Schemaverwaltung zuständig
- Nachteile
  - Schemalos (implizites Schema): Datenredundanz, keine JOINS, keine referentielle Integrität
- Systeme





… und viele mehr …

NoSQL-Datenmodelle
Document Stores - MongoDB



- **Insert** JSON documents
  - ```
    db.posts.insert( {
    _id : ObjectId("4e77bb3b8a3e000000004f7a"),
    when : Date("2011-09-19T02:10:11.3Z",
    author : "Altmann",
    title : "No Free Lunch",
    text : "This is the post text. It could be long.",
    tags : [ "business", "ramblings" ]
    })
    ```
- **Query** JSON documents
  - ```
    db.posts.find( { author : "Altmann" } )
    ```

NoSQL-Datenmodelle

Document Stores - MongoDB



- Besonderheiten
  - Skalierung durch Auto-Sharding und Master-Slave-Replikation
  - Fast relationale Anfragemöglichkeiten
  - Key-Value mit JSON-Dokumenten
  - Native Treiber für viele Programmiersprachen; REST
- Stärken
  - Umfangreiche Abfragefunktionalität
  - Tuning-Möglichkeiten
  - Changes API zur Überwachung von Änderungen
  - JSON Schema Validation
- Schwächen
  - Globales Locking kann Performance-Engpass bilden
  - Cluster-Betrieb erfordert Administrationsaufwand
- Einsatzgebiete
  - Skalierbare, vielseitig einsetzbare und benutzbare Ablage von objektorientierten Strukturen in JSON
  - Geeignet für kleine und große Problemstellungen

© 2026

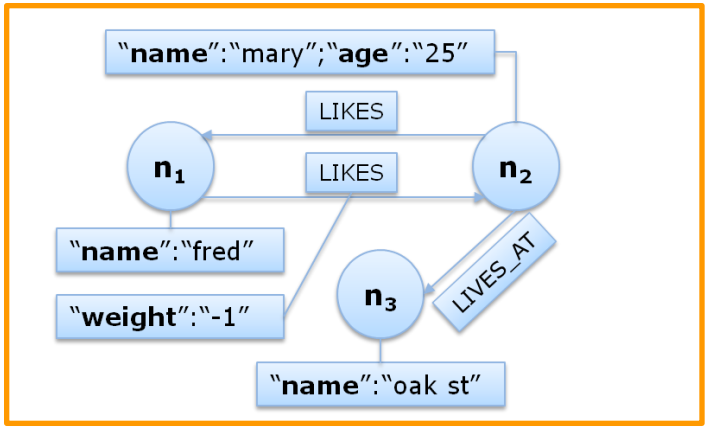
NoSQL-Datenmodelle
Document Store

- **When to Use**
  - If **many variants** of objects exist that need for **specific fields** → in RDBMS this would result in many `null` values or joins
  - If **data evolves heavily** → in RDBMS this would result in many schema changes
  - **Examples**
    - **Event Logging**: applications within an enterprise have different logging needs, which might change often
    - **E-Commerce Applications**: often need flexible schemas for products and orders
- **When Not to Use**
  - **Complex transactions** spanning different operations
    - Atomic cross-document operations are typically not supported

NoSQL-Datenmodelle
Graph Stores

- **Property-Graphen** bestehend aus Knoten und Kanten
  - Typisches Anwendungsfeld → Social network graphs
- **Interface**
  - `create: id`
  - `get(id)`
  - `connect(id1, id2): id`
  - `addAttribute(id, name, value)`
  - `getAttribute(id, name): value`

`id` der Kante



- **Datenspeicherung**
  - Daten in Knoten und typisierten Kanten abgelegt
  - Knoten und Kanten können Attribute enthalten
- Operationen zur **Pfadnavigation**, **Pfadsuche**, etc.

NoSQL-Datenmodelle
Graph Stores

- Vorteile
  - Kein starres Schema; Strukturänderungen einfach durchführbar
  - Ideal für die Lösung von Graph-Problemen
- Systeme













ALLEGROGRAPH

… und viele mehr …

NoSQL-Datenmodelle
Graph Stores – Neo4J

- Basic Operations

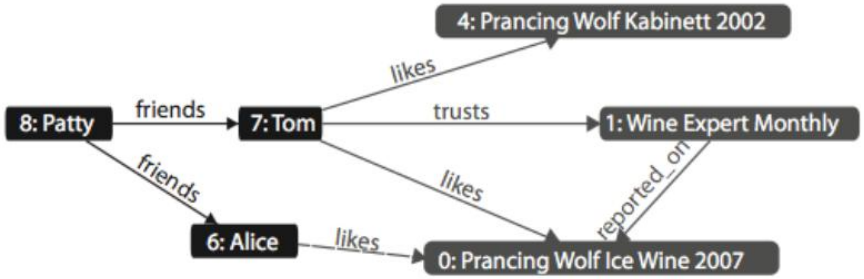
```
Graph graph = new Neo4jGraph("/tmp/my_graph");

Vertex a = graph.addVertex(null);
Vertex b = graph.addVertex(null);

a.setProperty("name","Patty");
b.setProperty("name","Tom");

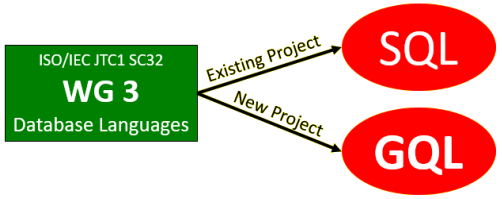
Edge e = graph.addEdge(null, a, b, "friends");
e.setProperty("since", 2001);

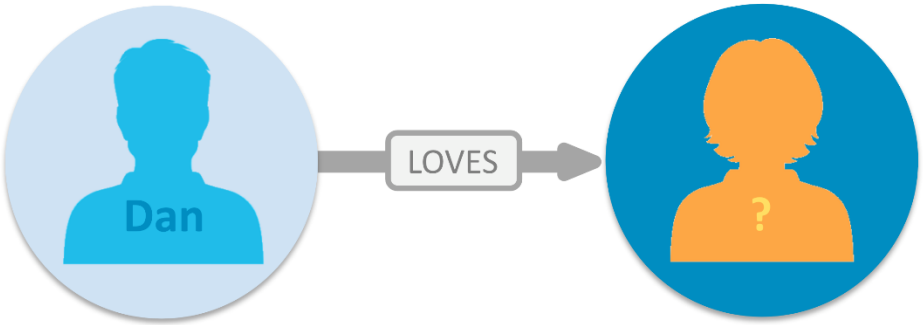
graph.shutdown(); //commit
```



NoSQL-Datenmodelle

Graph Stores – GQL (Graph Query Language)





NODE — Relationship — NODE

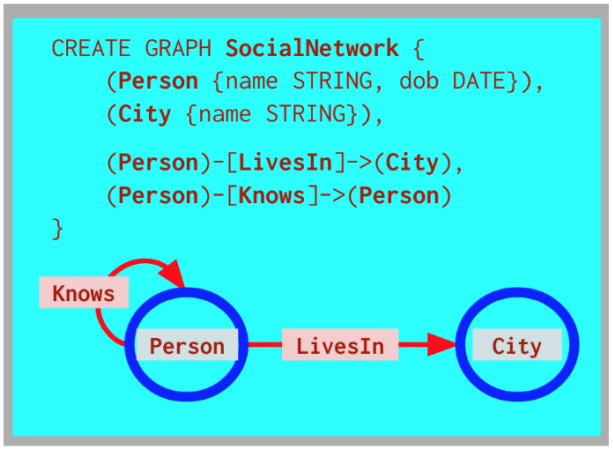
```
MATCH (:Person { name:"Dan"} ) -[:LOVES]-> ( whom ) RETURN whom
```

LABEL — PROPERTY — VARIABLE

NoSQL-Datenmodelle

Graph Stores – GQL (Graph Query Language)

Inspired by
- Cypher
- SQL/PGQ
- SPARQL



WG3:JCJ-015
DM32.2-2019-00999

ISO/IEC JTC 1/SC 32

Date: 2019-05-09

IWD 39075:202y(E)

ISO/IEC JTC 1/SC 32/WG 3

The United States of America (ANSI)

Information technology — Database languages — GQL

Technologies de l'information — Langages de base de données — GQL

SQL/PGQ querying – Example

Get the creationDate and content of the messages created by one person ("email1") and commented on by another person ("email2").

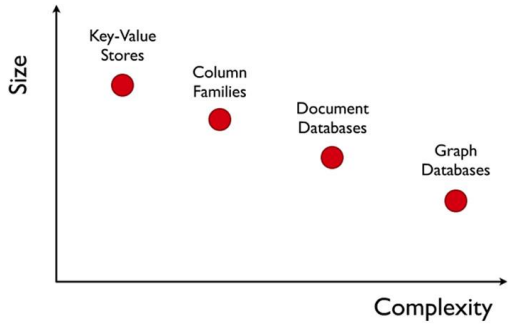
```
SELECT GT.creationDate, GT.content
FROM myGraph GRAPH_TABLE (
  MATCH
   (Creator IS Person WHERE Creator.email = :email1)
      -[ IS Created ]->
   (M IS Message)
      <-[ IS Commented ]-
   (Commenter IS Person WHERE Commenter.email = :email2)
      WHERE ALL_DIFFERENT (Creator, Commenter)
  COLUMNS (
      M.creationDate,
      M.content )
 ) AS GT
```

NoSQL-Datenmodelle
Graph Stores

- **When to Use**
  - If **data** is **highly connected**
  - **Example**
    - Social network graphs
    - Semantic Web
  - If **queries exploit** the **connectivity** of data
  - **Example**
    - Recommendation engines (“your friends also bought this product” or “when invoicing this item, these other items are usually invoiced too”)
- **When Not to Use**
  - If **global graph operations** are needed
    - e.g., all entities need to be updated with a changed property

© 2026

NoSQL-Datenmodelle
Produkte



Key-Value

Key	Value
1	Bob
2	Sue
3	Joe
4	Jo

Linked in
Project Voldemort
A distributed database.

amazon web services
Amazon DynamoDB

redis

Wide-Column

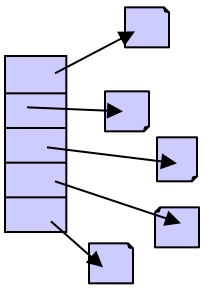
	Col 1	Col2	Col3	Col4	Col5	Col6	Col7	Col8
Row 1								
Row 2								
Row 3								
Row 4								
Row 5								
Row 6								
Row 7								
Row 8								

Google Bigtable

facebook
HBASE

twitter
Cassandra

Document



eXist
SimpleDB
apache CouchDB relax
powered by mongoDB

Graph-based

Neo4j
*dex
orient
R D F
redis
InfiniteGraph
AllegroGraph® RDFStore
FRANZ INC

cf: T. Haughey, The Role of Data Architecture in NOSQL, DAMA Day NY, Sept. 2011

NoSQL-Datenmodelle
Zusammenfassung

	Base Model	Flexibility of Structure	Complexity of DB	Scalability of Volume	Performance
Key-Value Stores	Associative array	High	None	High	High
Wide Column Stores	Key and column	Moderate	Low	High	High
Document Stores	Hierarchical	High	Low	Variable (High)	High
Graph Databases	Graph theory	High	High	Variable	Variable
Relational Databases	Relational algebra	Low in structure, but high in joining	Moderate	Variable	Variable

© 2026

Eigenschaften von NoSQL-Datenbanken

- Nicht-relational
- „Kein explizites Schema“
  - Hinzufügen von Attributen zur Laufzeit

→ **NoSQL-Datenmodelle** ✓

- Verfügbarkeit
  - Fehlertoleranz durch Replikation
- Horizontale Skalierbarkeit
  - Verteilte Architektur unterstützt Datenreplikation

→ **Skalierbarkeit** ✓

- Abstriche bei der Konsistenz
  - Eventually Consistency
  - BASE anstatt ACID

→ **Konsistenz**

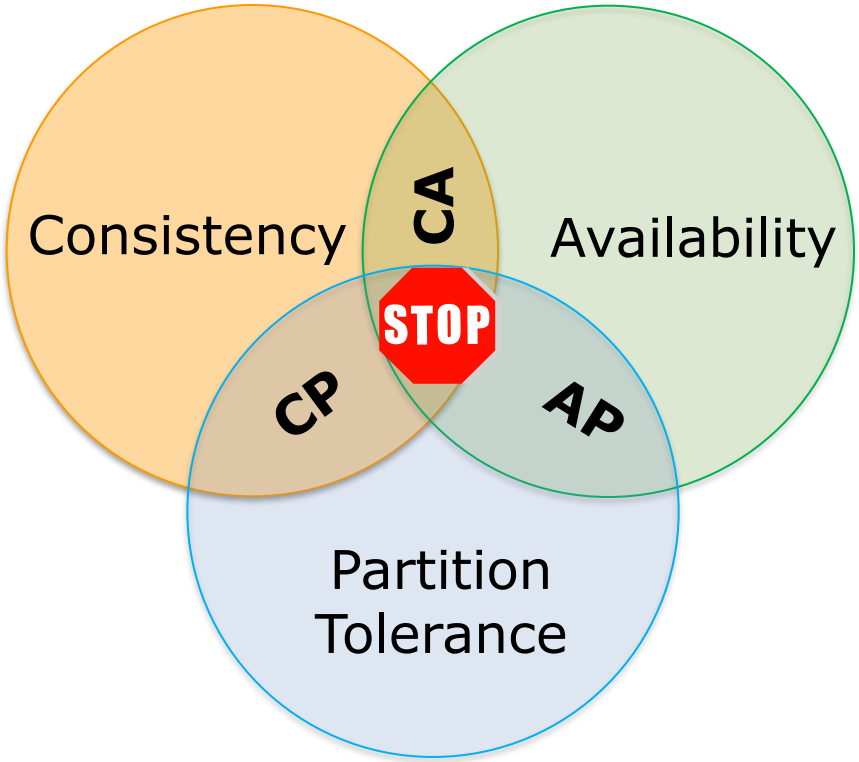
- Einfache API für Zugriff
  - keine JOINs

→ **MapReduce**

- Open Source

CAP-Theorem
in verteilten Datenbanksystemen

Daten zu jedem Zeitpunkt identisch in allen Replikaten



System ist jederzeit verfügbar und antwortet performant, d.h. auch: es kann trotz Knotenausfalls oder Netzwerkstörung geschrieben werden

Störung des Netzwerks oder Nachrichtenverlust führt nicht zu Stillstand des Gesamtsystems



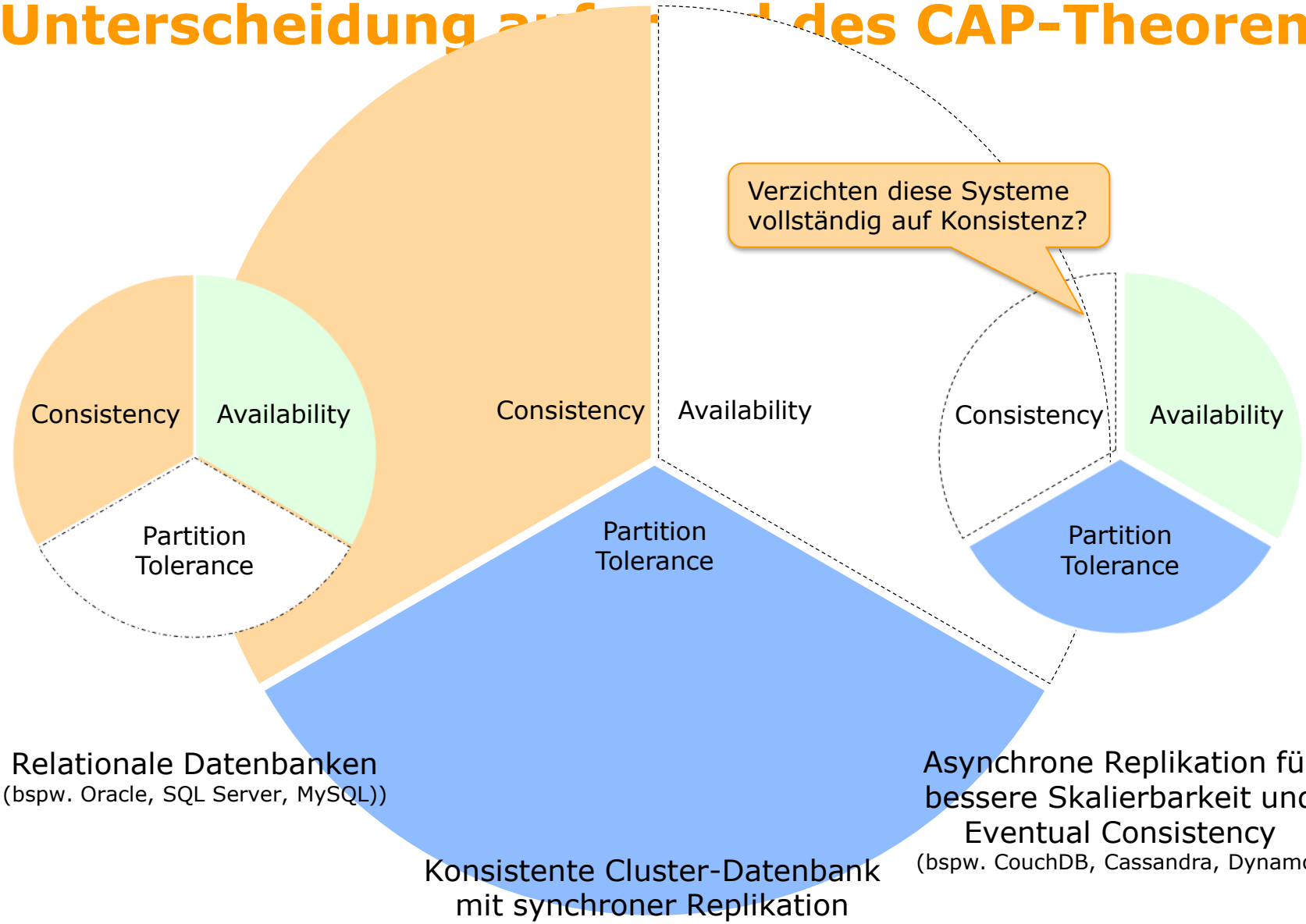
In verteilten Systemen können immer nur zwei der drei Eigenschaften gewährleistet werden!



Verzicht auf ACID-Transaktionen zugunsten von Skalierbarkeit!

E. A. Brewer, Towards robust distributed systems, Keynote PODC 2000
http://www.cs.berkeley.edu/~brewer/cs262b-2004/PODC-keynote.pdf

Unterscheidung auf Grund des CAP-Theorems

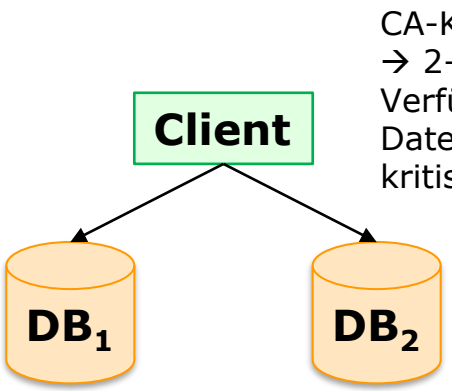


M. Kleppmann, CAP is broken, and it's time to replace it, Keynote Strange Loop 2015
https://martin.kleppmann.com/2015/09/25/cap-theorem-at-strange-loop.html

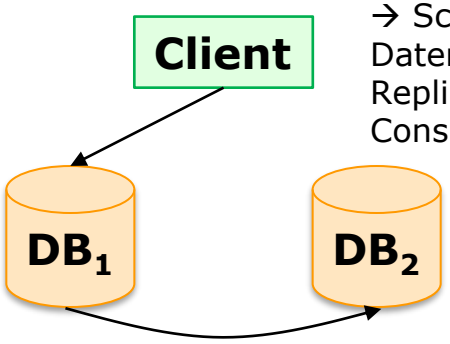
Konsistenzmodelle

Starke versus schwache Konsistenz

- **ACID**
  - **A**tomicity, **C**onsistency, **I**solation, **D**urability
  - Starke Konsistenz
    Domänen- und Entitätsintegrität, Referentielle und benutzerdefinierte Integrität, semantische und operationale Integrität, Replikationsintegrität
  - Pessimistisch
    - von vornherein
  - Verfügbarkeit leidet
  - Aufwändig zu implementieren:
    Sperren, 2PC, etc.
- **BASE**
  - **B**asically **A**vailable **S**oft-state **E**ventually consistent
  - Schwache Konsistenz
    - man nimmt Lost Update in Kauf
  - Optimistisch
    - späte Konfliktlösung
  - Schnell und einfach



CA-Konfiguration
→ 2-Phasen-Commit erfordert Verfügbarkeit von beiden Datenbanken; Latenz ist kritisch



AP-Konfiguration
→ Schreiben auf einen/wenige Datenbanken mit asynchroner Replikation für Eventual Consistency

BASE

Basically Available Soft-state Eventually consistent

- BASE schwächt Konsistenz und Isolation ab und fokussiert auf **Verfügbarkeit** und **Performanz**
- **Basically Available**
  - Application works all the time
- **Soft-State**
  - Es ist nicht garantiert, dass nachfolgende Zugriffe den aktuellen Wert sehen
  - *Readers may face inconsistent data*
- **Eventual consistency**
  - Es ist garantiert, dass nach einem Zeitfenster letztendlich (eventually) alle Zugriffe den aktuellen Wert sehen (falls zwischenzeitlich keine weiteren Updates erfolgen)
  - *Readers see writes as time goes on*

Eventual Consistency
Formen

- Read-your-writes Consistency
  - Jeder Prozess sieht seine eigenen Änderungen (niemals ältere Werte für die geänderten Objekte)
- Session Consistency
  - Read-your-writes innerhalb einer Session
- Monotonic Read Consistency
  - Wenn ein Prozess einen Wert gelesen hat, sieht er danach nie einen älteren Wert für dieses Objekt.
- Monotonic Write Consistency
  - Die Schreiboperationen einer Transaktion werden vom System serialisiert

- Konsistenzanforderungen der Anwendung bestimmen die Auswahl einer geeigneten NoSQL-Datenbank.
- Wenn das NoSQL-Datenbank die geforderten Merkmale nicht bietet, dann obliegt es dem Entwickler, diese selbst in der Anwendung zu programmieren.

Eigenschaften von NoSQL-Datenbanken

- Nicht-relational
- „Kein explizites Schema“
  - Hinzufügen von Attributen zur Laufzeit

→ **NoSQL-Datenmodelle**

- Verfügbarkeit
  - Fehlertoleranz durch Replikation
- Horizontale Skalierbarkeit
  - Verteilte Architektur unterstützt einer Datenreplikation

→ **Skalierbarkeit**

- Abstriche bei der Konsistenz
  - Eventually Consistency
  - BASE anstatt ACID

→ **Konsistenz**

- Einfache API für Zugriff
  - keine JOINs

→ **MapReduce**

- Open Source

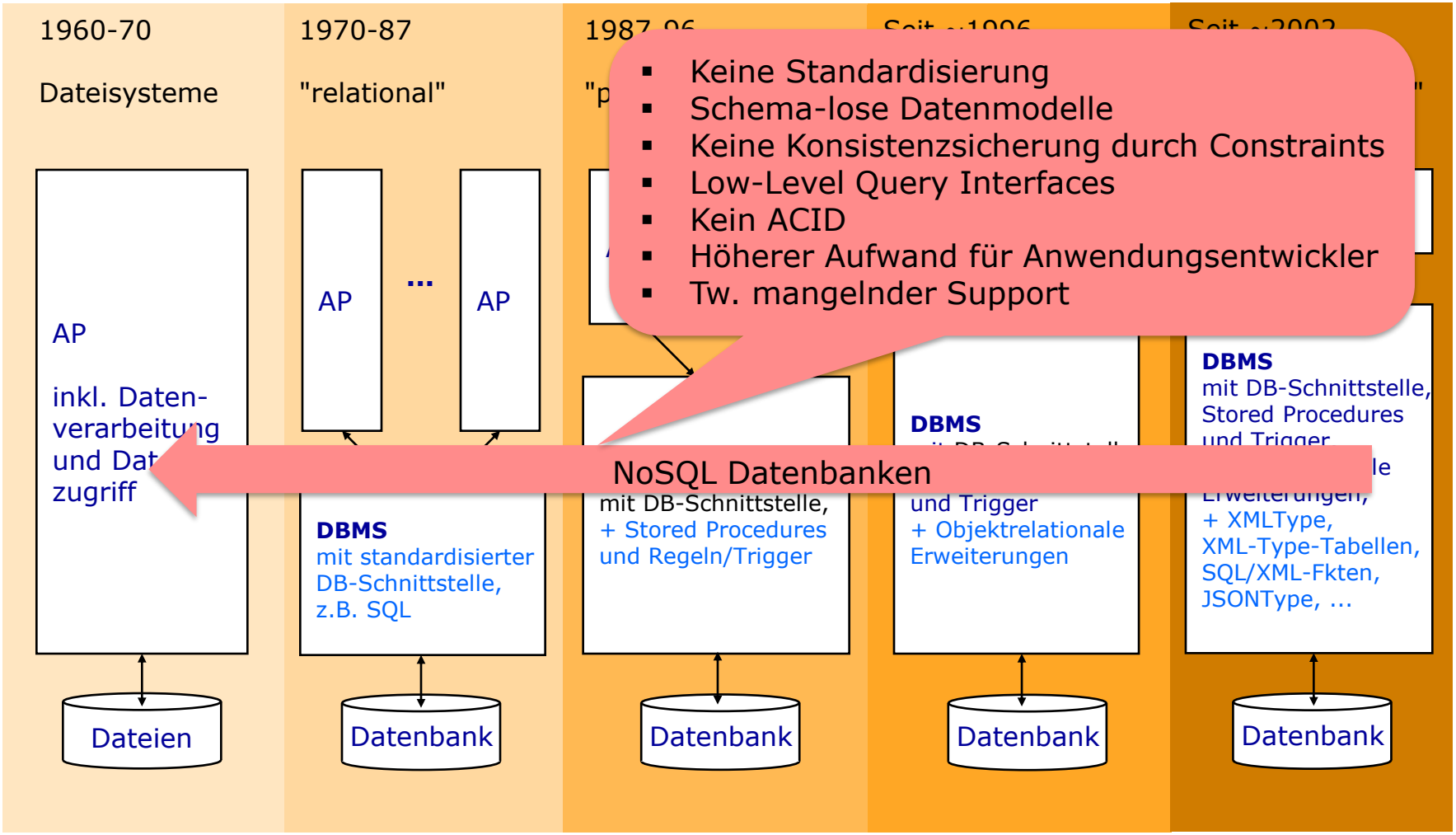
Überblick

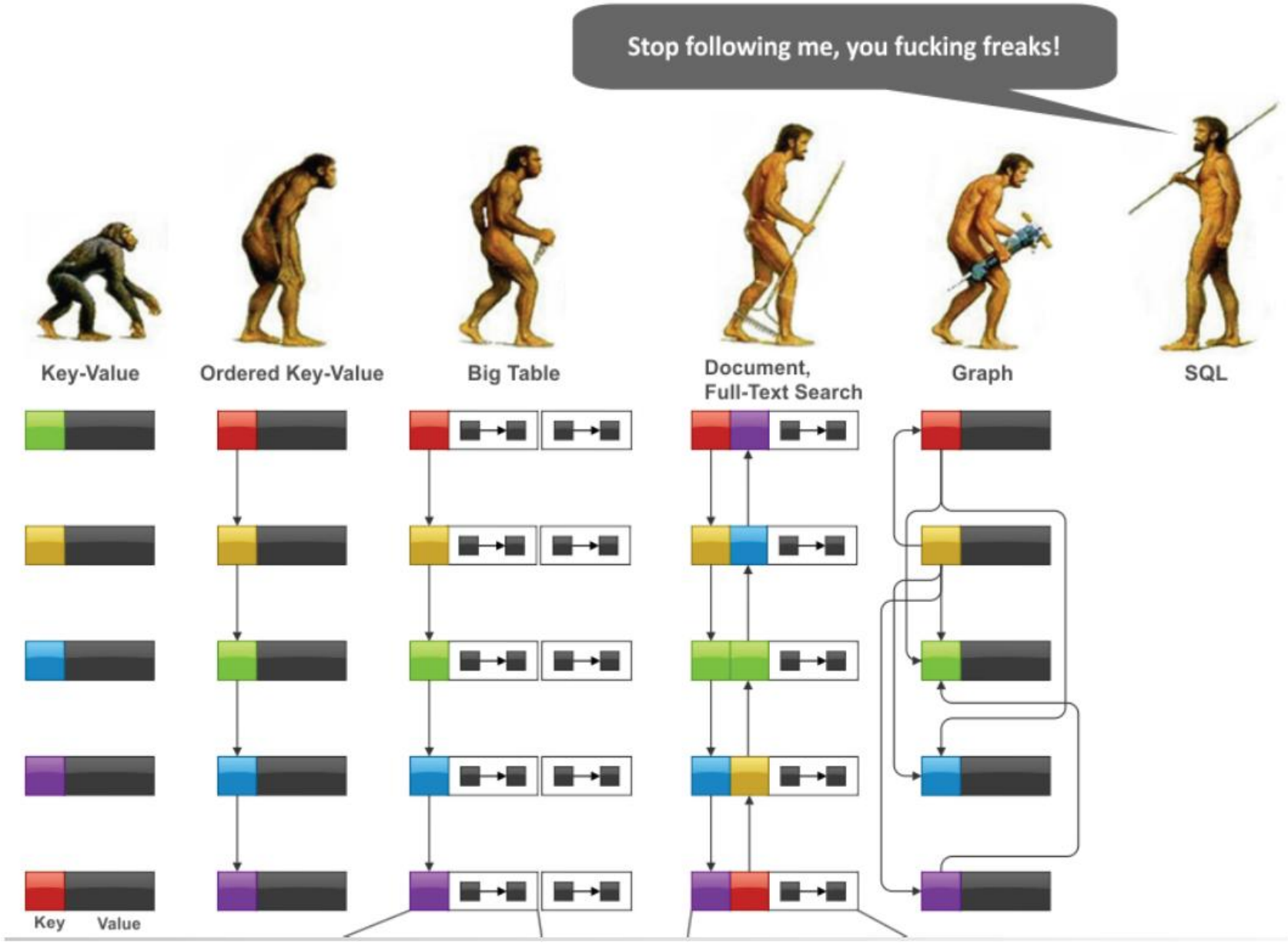
- Motivation
- Big Data
- NoSQL
- Zusammenfassung

Zusammenfassung
RDBMS versus NoSQL

RDBMS	NoSQL
Relationales Datenmodell (Schema-first) ▪ Starres Schema mit Integritätsbedingungen (Schlüssel, referentielle Integrität, etc.) - explizites Schema im Data Dictionary abgebildet ▪ Normalisierung um Redundanz zu vermeiden	**Nicht-relationale Datenmodell (Instance-first)** ▪ "Schema-los" oder nur schwache Schemarestriktionen – implizites Schema ▪ Keine Normalisierung, Redundanz tw. beabsichtigt
Relationale Datenabfrage und -manipulation ▪ Deklarative, normierte Abfragesprache (SQL) ▪ Automatische Abfrageoptimierung ▪ Indizes zur Verbesserung der Abfragegeschwindigkeit ▪ Gespeicherte Prozeduren und Trigger	**Alternative Datenmodelle mit API-Zugriff** oder anderen nicht SQL-basierten Zugriff (bspw. keine Joins) ▪ MapReduce
ACID ▪ (Strenge) Konsistenz und Sicherheit	**BASE** ▪ Eventual consistency
Optimiert für zentrales Deployment	**Horizontale Skalierbarkeit** durch horizontale Partitionierung
Typischerweise hohe Lizenzkosten	**Open Source**

Evolution von RDBMS




Stop following me, you fucking freaks!
Key-Value
Ordered Key-Value
Big Table
Document,
Full-Text Search
Graph
SQL
Key
Value

NoSQL als Hype?

- "NoSQL databases will be used largely for working with **unstructured data** in ways that require **scalability**."
  *by **Dave Rosenberg**, founder of open source infrastructure provider MuleSource*

- "NoSQL adoption will be small-scale and only in **some niches** because **relational databases** are more **mature** and represent **huge investments** by vendors and users."
  *by **Anant Jhingran**, IBMs chief technology officer*

- "People will learn to look at their data and select **many databases** for **many needs**."
  *by **Stefan Edlich**, professor at the Beuth University of Applied Sciences in Berlin*

Polyglot Persistence

NoSQL als Hype?

"Will there be some sort of "war" between NoSQL and SQL supporters like the one of REST versus SOAP? The answer is maybe. Who will win this case? As with SOAP versus REST, there won't be a winner or a loser. ***We will have more opportunities to choose our database systems in the future.*** *For data warehousing and systems that require business intelligence to be in the database, SQL databases might be your choice. If you need high-responsive, scalable and flexible databases, NoSQL might be better for you. "*

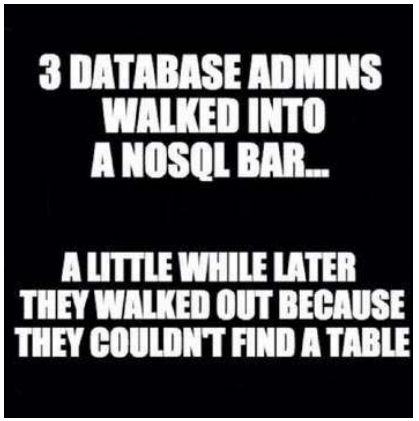
[Mario Meir-Huber: NoSQL – The Trend for Databases in the Cloud? http://soa.sys-con.com/node/1615716]

Zusammenfassung
NoSQL

- Schlanke, effiziente Datenbanken, die auf die kostengünstige Verwaltung einfach strukturierter, sehr großer Datenmengen (Big Data) spezialisiert sind
- Verzicht auf strenge Konsistenz und ausgeprägte Schema-Strukturen ermöglicht performantere und ausfalltolerantere Datenverteilung

- ABER:
  - Skalierbarkeit hat ihren Preis: Kosten oder Funktionalität
  - Fehlende Funktionalität (Transaktionsunterstützung, Anfragesprache mit optimierten Ausführungsplänen und Zugriffsalgorithmen, Verbundoperationen, etc.) muss bei Bedarf in der Anwendung nachimplementiert beiden
  - Eventuelle Konsistenz
  - NoSQL-Skillgap
  - NoSQL-Lock-in: Migration = Nightmare
  - Volatiler Markt

Zusammenfassung
RDBMS

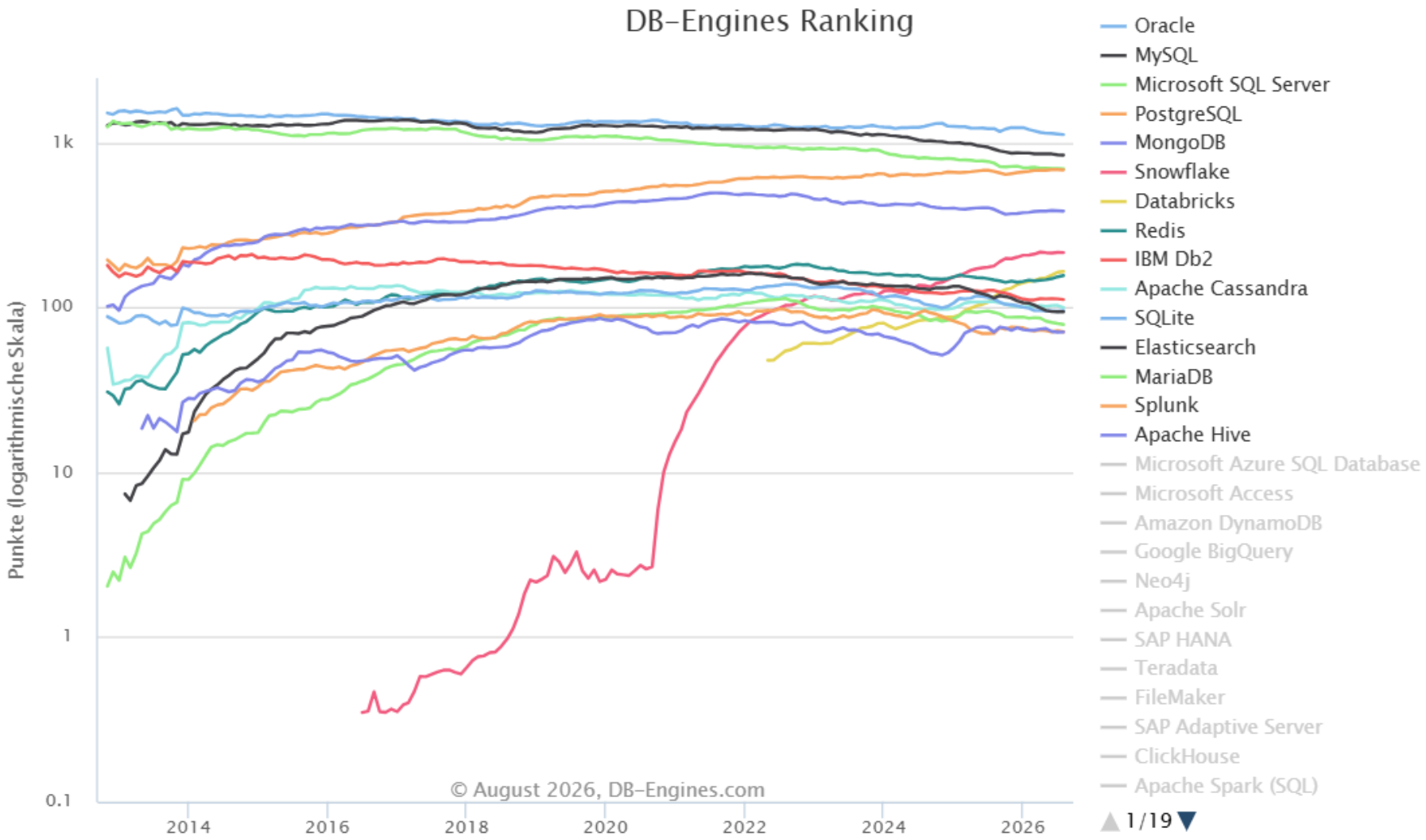


- Relationale Datenbanken bleiben Standard
  - Strukturierte Daten sowieso
  - In-Memory für Echtzeit-Analysen
  - Integration von Hadoop als intelligentem Storage
  - Integration von JSON für flexibles Schema
  - Ausbau der Cluster-Fähigkeiten
- Etablierte relationale Datenbanken integrieren ähnliche Funktionen (bspw. Oracle/Key Value, Microsoft/Cosmos DB)
- Polyglot Persistence

➢ Haben NoSQL-Datenbanken eine eigenständige Überlebenschance oder ereilt sie das gleiche Schicksal wie OODBMS und reinen XML-DBMS????

Zusammenfassung

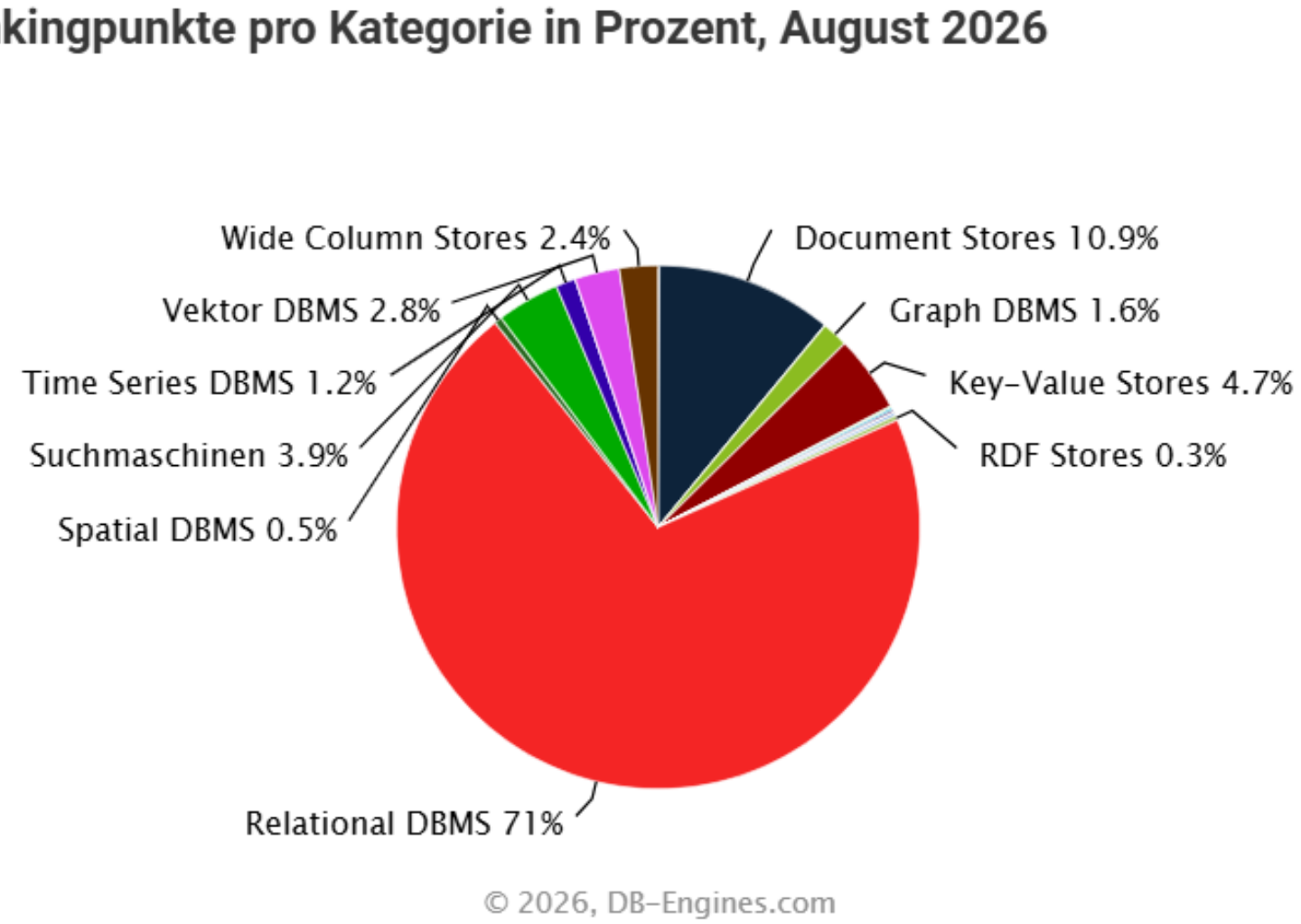
„Popularität“ von Datenbanksystemen



https://db-engines.com/de/ranking_trend

Zusammenfassung

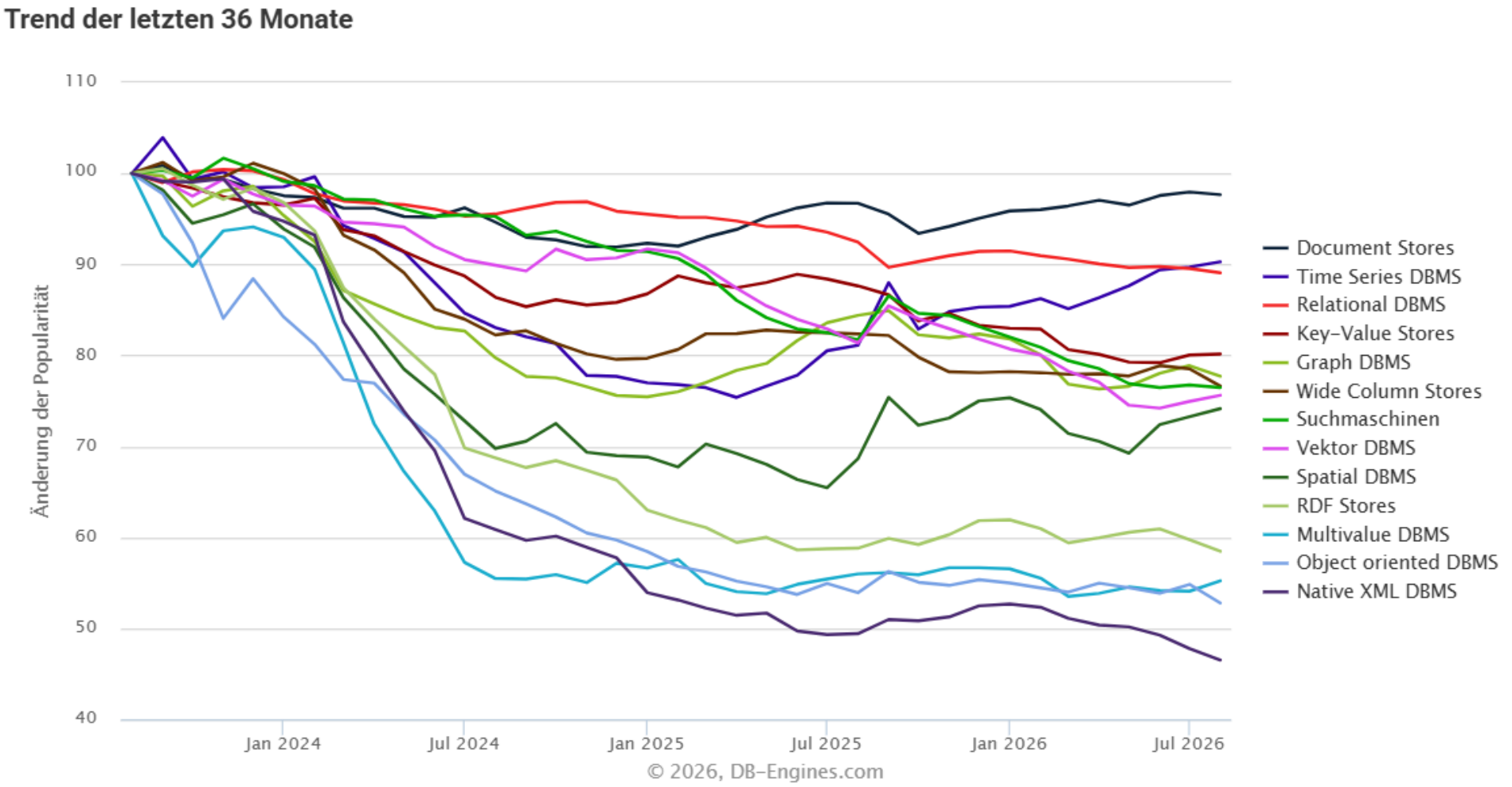
„Popularität“ von Datenbankmodellen



https://db-engines.com/de/ranking_categories

Zusammenfassung

„Popularität“ von Datenbankmodellen



https://db-engines.com/de/ranking_trend

Google Cloud Spanner

> We believe it is better to have application programmers deal with performance problems due to overuse of transactions as bottlenecks arise, rather than always coding around the lack of transactions.
>
> **Google 2012**

- Google Cloud Spanner
  - Klassische relationale Datenbank mit SQL und strikten ACID-Transaktionen
  - Horizontale Skalierbarkeit
  - Automatische Replikation
  - Basis für Google AdWords

➢ **Skalierung geht auch relational im großen Stil!**

J. Shute, R. Vingralek, B. Samwel, B. Handy, C. Whipkey, E. Rollins, M. Oancea, K. Littlefield, D. Menestrina, S. Ellner, J. Cieslewicz, I. Rae, T. Stancescu, H. Apte. 2013. F1: A Distributed SQL Database That Scales. VLDB, Vol. 6, No. 11

Entscheidung für oder gegen NoSQL-Datenbank

- **DB Jungle Guide** – „How to select the right database”

  http://nosql-database.org/select-the-right-database.html

- **Six categories** with 70 questions
  - Know & segment your data
  - Consistency model
  - Performance dimensions
  - Query requirements
  - Architecture and patterns
  - Non-functional requirements



"It was much nicer before people started storing all their data in the Cloud."

New Yorker Cartoon by Mick Stevens (http://www.mickstevens.com)

© 2026

Anhang I: Definitionen

Begriff	Bedeutung
Sharding	Sharding ist eine Methode der Datenbankpartitionierung. Dabei wird ein Datenbestand in mehrere Teile aufgeteilt und jeweils von einer eigenen Serverinstanz verwaltet. Z.B. Personen werden je nach Anfangsbuchstaben auf drei Server aufgeteilt: A-F, G-O, P-Z. Wann immer eine Person gespeichert oder über den Namen abgefragt wird, wird zuerst der zuständige Server bestimmt, an den der Request weitergeleitet wird. **Vorteile** Mittels Sharding können umfangreiche Datenmengen verwaltet werden, welche die Kapazitäten eines einzelnen Servers sprengen würden. Da die einzelnen Teile von einer eigenen Serverinstanz verwaltet werden, werden nicht nur die Daten selbst, sondern auch die dafür benötige Rechenleistung aufgeteilt. **Partitionsstrategie** Die Methode der Aufteilung, die sog. Partitionsstrategie, spielt dabei eine entscheidende Rolle. Das obige Beispiel der Aufteilung nach Anfangsbuchstaben hat den Vorteil, dass auch Abfragen sortiert nach dem Aufteilungskriterium leicht möglich sind. Ein gravierender Nachteil ist allerdings, dass das Hinzufügen weiterer Server umfangreiches Umspeichern erfordert. Daher werden, vor allem in Szenarien mit Dutzenden oder Hunderten Server, eher Hash-basierte Verfahren verwendet. Dabei kommen sog. konsistente Hash-Funktionen zum Einsatz, welche die Anzahl der in diesen Fällen notwendigen Umspeicherungen minimieren.

© 2026

Anhang I: Definitionen

Begriff	Bedeutung
Sharding	**Nachteile und Restriktionen** Der größte Nachteil des Shardings ist natürlich, dass ein Zugriff über andere Kriterien als das Aufteilungskriterium unverhältnismäßig aufwändig ist. Abfragen über andere Kriterien oder Joins müssen i.A. auf alle Server aufgeteilt werden. Das Datenmodell und die Zugriffspfade müssen so entworfen werden, dass derartige Zugriffe nur selten oder gar nicht vorkommen, sonst werden die Vorteile des Shardings zunichte gemacht. Diese Notwendigkeit macht Sharding zu einer attraktiven Methode vor allem für NoSQL-Systeme, bei denen typischerweise Joins ohnehin nicht unterstützt werden, und auch Zugriffe über Sekundärschlüssel eher eine Ausnahme sind. Somit lässt sich Sharding für diese Systeme vergleichsweise einfach implementieren. Man kann sogar sagen, dass das relativ problemlose Sharding einer der Hauptgründe für den rasch steigenden Einsatz von NoSQL-Systemen ist. **Sharding und Replikation** Bei mehreren NoSQL-Systemen wird Sharding und Replikation auf elegante und effizient Weise verknüpft, etwa bei Cassandra und Riak. Dabei wird z.B. in einer ringförmigen Anordnung der Server jeder Shard auch gleichzeitig auf einen oder mehrere nachfolgende Server im Ring repliziert. Somit hält jeder Server jeweils eine Kopie mehrerer Shards. Durch diese Anordnung und die dazugehörigen komplexen Datenabgleichsmechanismen entstehen sehr flexible und fehlertolerante Systeme, welche auch mit sehr großen Datenmengen befüllt werden können.

© 2026